BK Restaurant Management System

Software Design Specification

Course	SENG 31242 – System Design Project
Group	01
Team Members	SE/2022/020 – Sachin Lakshitha SE/2022/023 – Duvini Nimethra SE/2022/026 – Dinuja Ranaweera SE/2022/030 – Nethmi Navodya
Client	Mr. N. C. Sanjeewa De Silva, BK Restaurant & BK Bar
Related Document	Software Requirements Specification (SRS) – BK Restaurant Management System
Document Status	Draft

Contents

1. Introduction

1.1 Purpose

The purpose of this Software Design Specification (SDS) is to define the technical design of the BK Restaurant Management System in sufficient detail to guide its implementation, testing, and future maintenance. This document translates the requirements captured in the Software Requirements Specification (SRS) into a concrete architecture, component structure, data model, interface design, and security design. It is intended for use by the development team during implementation, by reviewers assessing design quality against the requirements, and by future maintainers extending the system.

1.2 Scope

This document covers the design of all major subsystems identified in the SRS, namely:

- The Point of Sale (POS) and Kitchen Display System (KDS) used by reception and kitchen staff
- The centralised backend API and database serving all branches
- The Flutter-based mobile application used by delivery riders
- The centralised web dashboard used by the Restaurant Owner for multi-branch monitoring and analytics
- The inventory management, supplier request, and employee management modules

The design described here directly addresses the Functional Requirements (FR-01 to FR-18) and Non-Functional Requirements (NFR-01 to NFR-22) defined in the SRS. Implementation-level detail such as exact source code, configuration files, and deployment scripts is outside the scope of this document and will be produced during the development phase.

1.3 References

- IEEE Std 1016–1998 – IEEE Recommended Practice for Software Design Descriptions.
- ISO/IEC/IEEE 42010:2011 – Systems and Software Engineering – Architecture Description.
- OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks. https://owasp.org/Top10/
- React.js Documentation. https://react.dev
- Node.js and Express.js Documentation. https://nodejs.org, https://expressjs.com
- PostgreSQL Documentation. https://www.postgresql.org/docs/
- Flutter Documentation. https://flutter.dev

1.4 Overview of Document

This document follows the SDS structure agreed for SENG 31242:

- Section 2 – System Overview: a summary of the system to be built and its relationship to the SRS.
- Section 3 – Architectural Design: the selected architecture pattern, justification, component/package diagram, deployment diagram, and technology stack.
- Section 4 – Detailed Design: class diagram, sequence diagrams (one per SRS use case), state machine diagrams for stateful entities, and the database/ER diagram.

- Section 5 – UI Design: wireframes for all major screens, the navigation flow diagram, and UX decisions and rationale.
- Section 6 – Interface Design: external interfaces (APIs, hardware, third-party services) and internal module interfaces.
- Section 7 – Security Design: authentication and authorisation strategy, data protection, input validation, and OWASP Top 10 considerations.
- Section 8 – Non-Functional Design Decisions: how each NFR from the SRS is addressed by the design.

2. System Overview

2.1 Summary of the System to be Built

The BK Restaurant Management System is a custom, web- and mobile-based platform that digitises and centralises operations for BK Restaurant and BK Bar across its branches in Ambalangoda and Kahawa, Sri Lanka. The system replaces the existing manual, WhatsApp- and paper-based workflow with an integrated set of modules:

- A web-based POS system for counter order entry, billing, and invoice generation.
- A Kitchen Display System (KDS) that shows incoming orders to kitchen staff in real time.
- A Flutter mobile application for delivery riders, providing assignment notifications, navigation, live GPS tracking, and delivery status updates.
- An inventory management module with automatic stock deduction, low-stock alerts, and digital supplier requests.
- An employee management module covering attendance, shift scheduling, and leave requests.
- A centralised owner dashboard for multi-branch monitoring, sales/revenue reporting, and delivery tracking analytics.

The system is built around a single backend API and a shared PostgreSQL database, with role-based access control ensuring that Restaurant Owners, Branch Managers, Reception/Kitchen Staff, and Delivery Riders each see only the features relevant to their role.

2.2 Relationship to the SRS

This SDS implements the requirements defined in the SRS as follows:

SRS Reference	How it is Addressed in this Design
FR-01, FR-02, FR-03, FR-04 (Order Management, Billing, KDS, Modification)	Implemented by the POS/KDS module, the Order and Invoice entities in the data model, and SSD-02/SSD-03 sequence diagrams (Section 4.2).
FR-05, FR-06, FR-07, FR-08 (Delivery Assignment and Tracking)	Implemented by the Delivery module, the rider mobile app, the GPS integration described in Section 6.1, and SSD-05/SSD-06/SSD-07 sequence diagrams.
FR-09, FR-10, FR-11 (Inventory and Supplier Requests)	Implemented by the Inventory module, the Inventory Item and Stock Request entities, and the SSD-09 sequence diagram.
FR-12, FR-13, FR-14 (Employee Attendance, Leave, Scheduling)	Implemented by the Employee Management module and the SSD-08 sequence diagram.
FR-15, FR-16 (Multi-Branch Dashboard and Reporting)	Implemented by the Owner Dashboard module and the SSD-10 sequence diagram.
FR-17, FR-18 (Role-Based Access Control, Secure Authentication)	Implemented by the security design in Section 7 and the SSD-01 sequence diagram.
NFR-01 to NFR-22 (Performance, Security, Usability, Reliability, Scalability, Maintainability, Compatibility, Availability)	Addressed individually in Section 8, with traceability to the corresponding design decision.

3. Architectural Design

3.1 Architecture Pattern Selected

The BK Restaurant Management System adopts a layered (n-tier) client–server architecture combined with a modular, feature-based organisation of the backend codebase. The system is structured into the following layers:

- **Presentation Layer** – the React.js web application (POS, KDS, and Owner Dashboard) and the Flutter mobile application (Rider App).
- **Application / API Layer** – a Node.js and Express.js REST API that exposes endpoints for each functional module (orders, inventory, delivery, employees, reporting, authentication).
- **Business Logic Layer** – service modules within the backend that implement validation rules, status transitions, and calculations (e.g., bill totals, inventory deductions, low-stock thresholds).
- **Data Access Layer** – an ORM-based data access layer that maps domain entities to the PostgreSQL database.
- **Data Layer** – a single PostgreSQL database shared across all branches, with Redis used as a caching layer for frequently accessed data such as menu items and dashboard summaries.

Within the Application Layer, the backend is further divided into independent modules (Authentication, Orders, Inventory, Delivery, Employees, Reporting), each exposing its own set of routes, controllers, and services. This gives the system the maintainability benefits of a modular/feature-based design while keeping the overall deployment simple enough for a small team to manage, which is appropriate given the project's scale and timeline.

3.2 Justification (Quality Attributes)

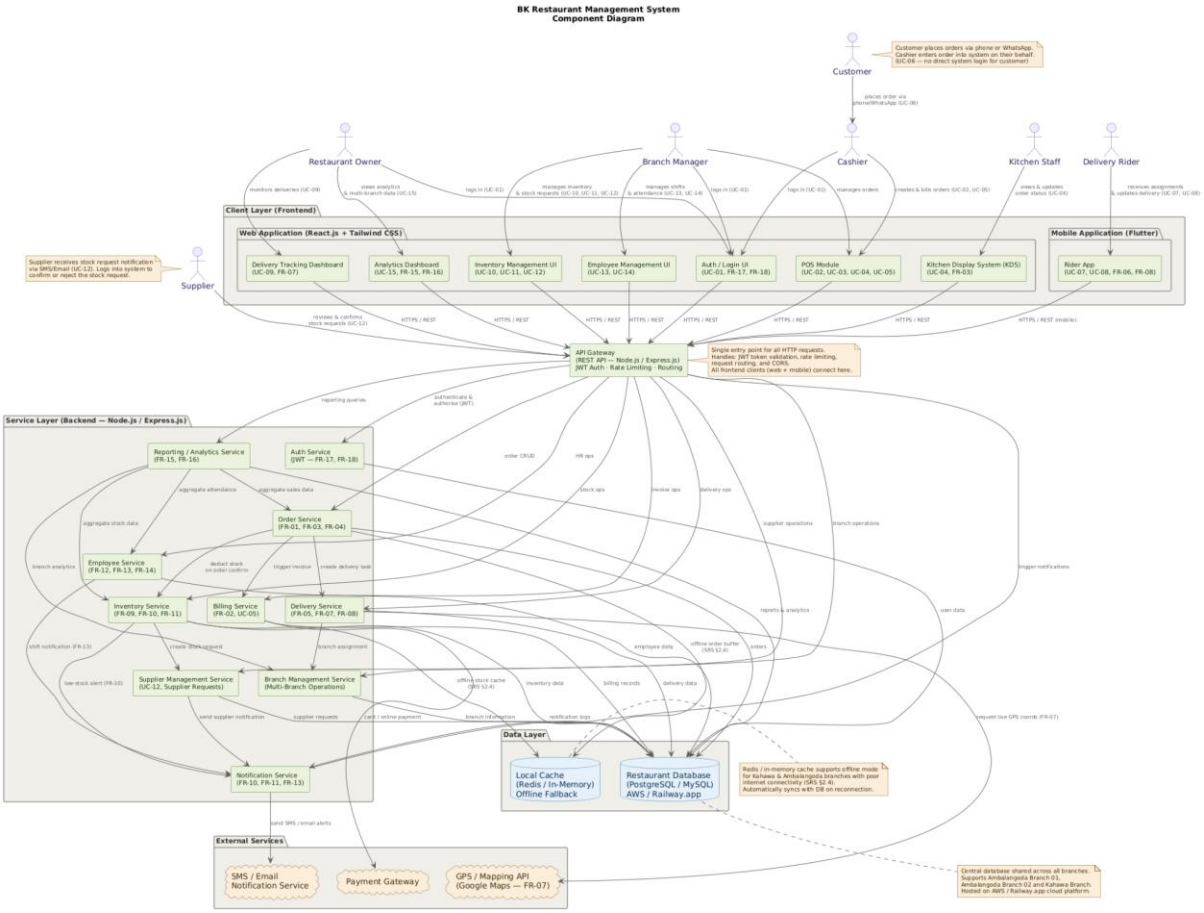
Quality Attribute	How the Architecture Supports It
Maintainability	Each functional module (orders, inventory, delivery, employees, reporting, authentication) is implemented as a separate package with its own routes, controllers, and services, allowing developers to modify one module without affecting others. This directly supports NFR-16 and NFR-17.
Scalability	The stateless API layer can be horizontally scaled behind a load balancer, and Redis caching reduces database load for read-heavy dashboard queries. New branches can be onboarded by adding records rather than changing the architecture, supporting NFR-14 and NFR-15.
Reliability	A layered design isolates business logic from data access, making it easier to add transaction handling, retries, and offline synchronisation for POS terminals and the rider app, supporting NFR-11, NFR-12, and NFR-13.
Performance	The API layer is kept thin and stateless, with caching for frequently read data (menu, dashboard summaries) to meet the 2-second response target in NFR-01 and to support concurrent multi-branch access (NFR-02).

Security	Centralising authentication and authorisation in a single module within the Application Layer (Section 7) ensures consistent enforcement of role-based access control across web and mobile clients, supporting NFR-04 to NFR-07.
Cost-Effectiveness	The chosen stack (React, Node.js/Express, PostgreSQL, Flutter) is open-source and well supported on low-cost cloud platforms, aligning with the design and implementation constraints identified in the SRS (Section 2.4).

3.3 Component / Package Diagram

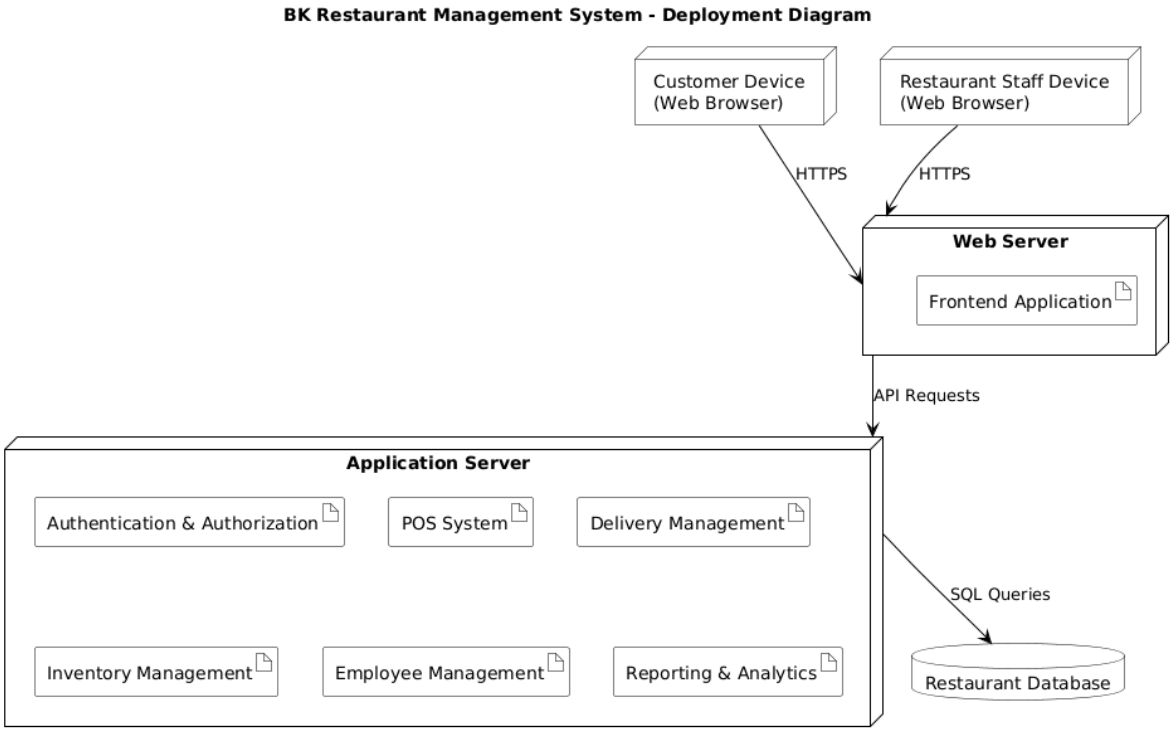
The Component Diagram illustrates the high-level architecture of the BK Restaurant Management System. It identifies the major system components, including the frontend applications, API Gateway, backend services, database, cache, and external services. The diagram shows how these components interact to support restaurant operations such as order processing, inventory management, delivery tracking, employee management, reporting, and authentication.

Figure 3.3.1: BK Restaurant Management System Component Diagram



3.4 Deployment Diagram

The deployment diagram below shows how the software components are mapped onto physical and virtual infrastructure nodes across the three restaurant branches and the cloud hosting environment.



3.5 Technology Stack with Justification

Layer / Component	Technology	Justification
Web Frontend (POS, KDS, Owner Dashboard)	React.js with Tailwind CSS	Mature, component-based library suited to building responsive POS and dashboard interfaces; Tailwind CSS speeds up consistent styling across screens (per SRS Section 2.3).
Mobile Frontend (Rider App)	Flutter	Cross-platform framework allowing a single codebase to target Android devices used by delivery riders, with good support for maps and GPS APIs.
Backend / API	Node.js with Express.js	Lightweight, widely adopted framework for building REST APIs; non-blocking I/O suits

		the real-time order and delivery update requirements (NFR-01, NFR-03).
Database	PostgreSQL	Reliable relational database suitable for transactional data such as orders, invoices, inventory, and employee records, with strong support for referential integrity across branches.
Caching	Redis	In-memory data store used to cache menu data and dashboard summaries, reducing database load and improving response times (NFR-01, NFR-02).
Authentication	JSON Web Tokens (JWT)	Stateless token-based authentication suitable for both the web and mobile clients, enabling role-based access control (FR-17, FR-18, NFR-04 to NFR-06).
Maps / GPS	Google Maps Platform (Maps SDK and Directions API)	Provides live location tracking and navigation for delivery riders and the owner's delivery monitoring dashboard (FR-07, UC-09).
Hosting	AWS Free Tier or Railway.app	Cost-effective cloud hosting options identified as economically feasible in the SRS feasibility study (Section 6.2.2).
Notifications	Email/SMS gateway (e.g., SendGrid, Twilio)	Used for sending invoices to customers and notifying suppliers of stock requests (FR-05, UC-12).

4. Detailed Design

4.1 Class Diagram

The class diagram below shows the core domain classes of the BK Restaurant Management System, their attributes, key methods, and the relationships between them. It forms the basis for the database schema described in Section 4.4.

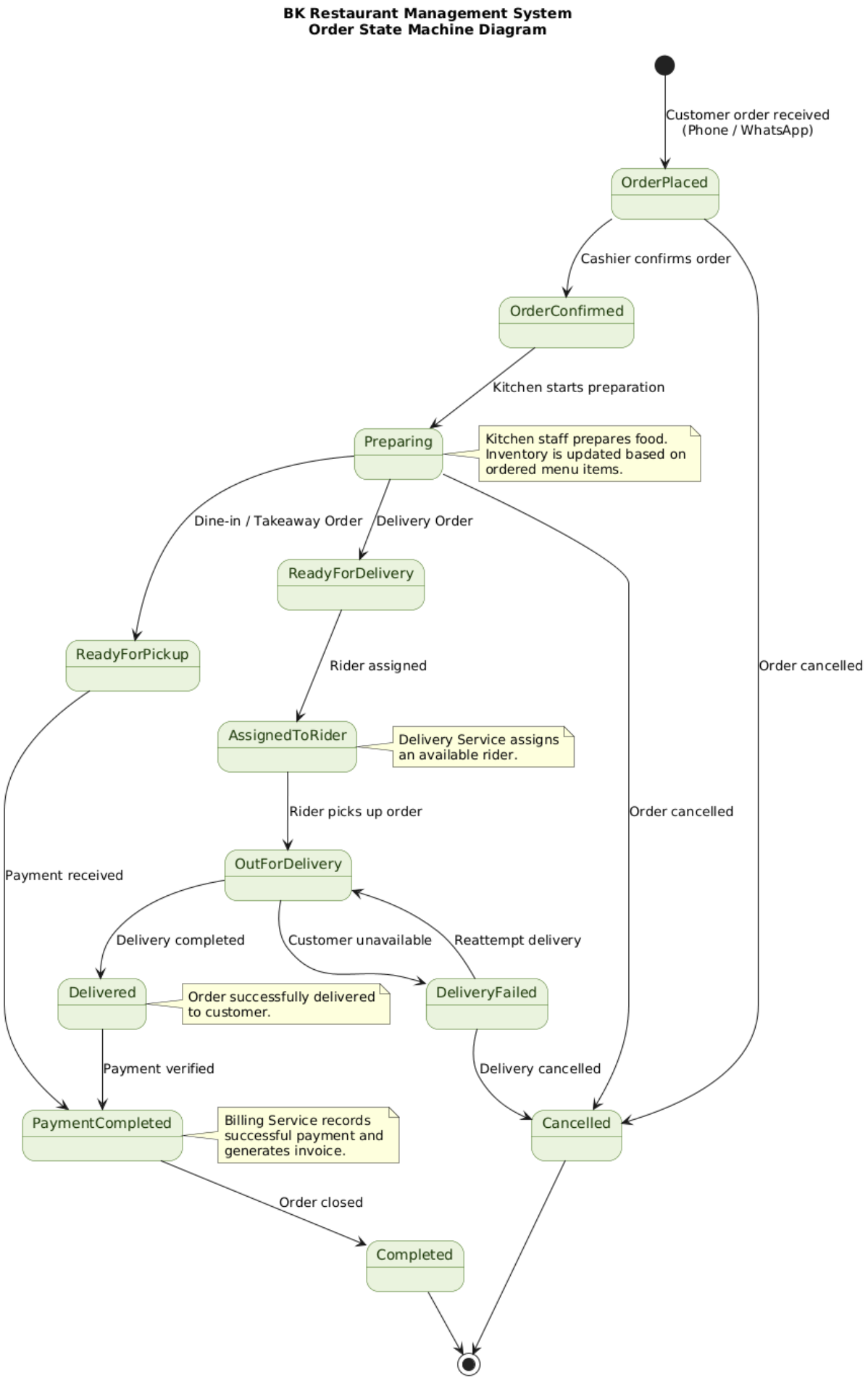
4.2 Sequence Diagrams

This sub-section provides one sequence diagram for each use case defined in the SRS (Section 3.4), showing the detailed message exchange between the actor, the client application, the API, and the database/services. Each diagram corresponds to a System Sequence Diagram (SSD) from the SRS, expanded to show internal system interactions.

4.3 State Machine Diagrams

The State Machine Diagram represents the lifecycle of an order within the BK Restaurant Management System. It shows the valid state transitions from order creation through preparation, delivery, completion, cancellation, and refund processing. The diagram ensures that order status changes follow the defined business rules and prevents invalid transitions between states.

Figure 4.3.1: BK Restaurant Management System Order State Machine Diagram


BK Restaurant Management System
Order State Machine Diagram
Customer order received
(Phone / WhatsApp)
OrderPlaced
Cashier confirms order
OrderConfirmed
Kitchen starts preparation
Preparing
Kitchen staff prepares food.
Inventory is updated based on
ordered menu items.
Dine-in / Takeaway Order
Delivery Order
ReadyForDelivery
ReadyForPickup
Rider assigned
AssignedToRider
Delivery Service assigns
an available rider.
Rider picks up order
OutForDelivery
Payment received
Delivery completed
Customer unavailable
Reattempt delivery
Delivered
Order successfully delivered
to customer.
DeliveryFailed
Payment verified
Delivery cancelled
PaymentCompleted
Billing Service records
successful payment and
generates invoice.
Cancelled
Order cancelled
Order cancelled
Order closed
Completed

4.4 Database / Data Model (Entity-Relationship Diagram)

The entity-relationship diagram below defines the relational database schema implementing the class diagram in Section 4.1. All entities are stored in a single shared PostgreSQL database, with a branchId foreign key used to partition data by branch where applicable.

Each row above maps directly to a table in the implementation database, with primary keys (PK) as auto-incrementing integers or UUIDs and foreign keys (FK) enforcing referential integrity. Indexes should be added on frequently queried columns such as Order.branch_id, Order.status, InventoryItem.branch_id, and Delivery.status to support the performance requirements in NFR-01 and NFR-02.

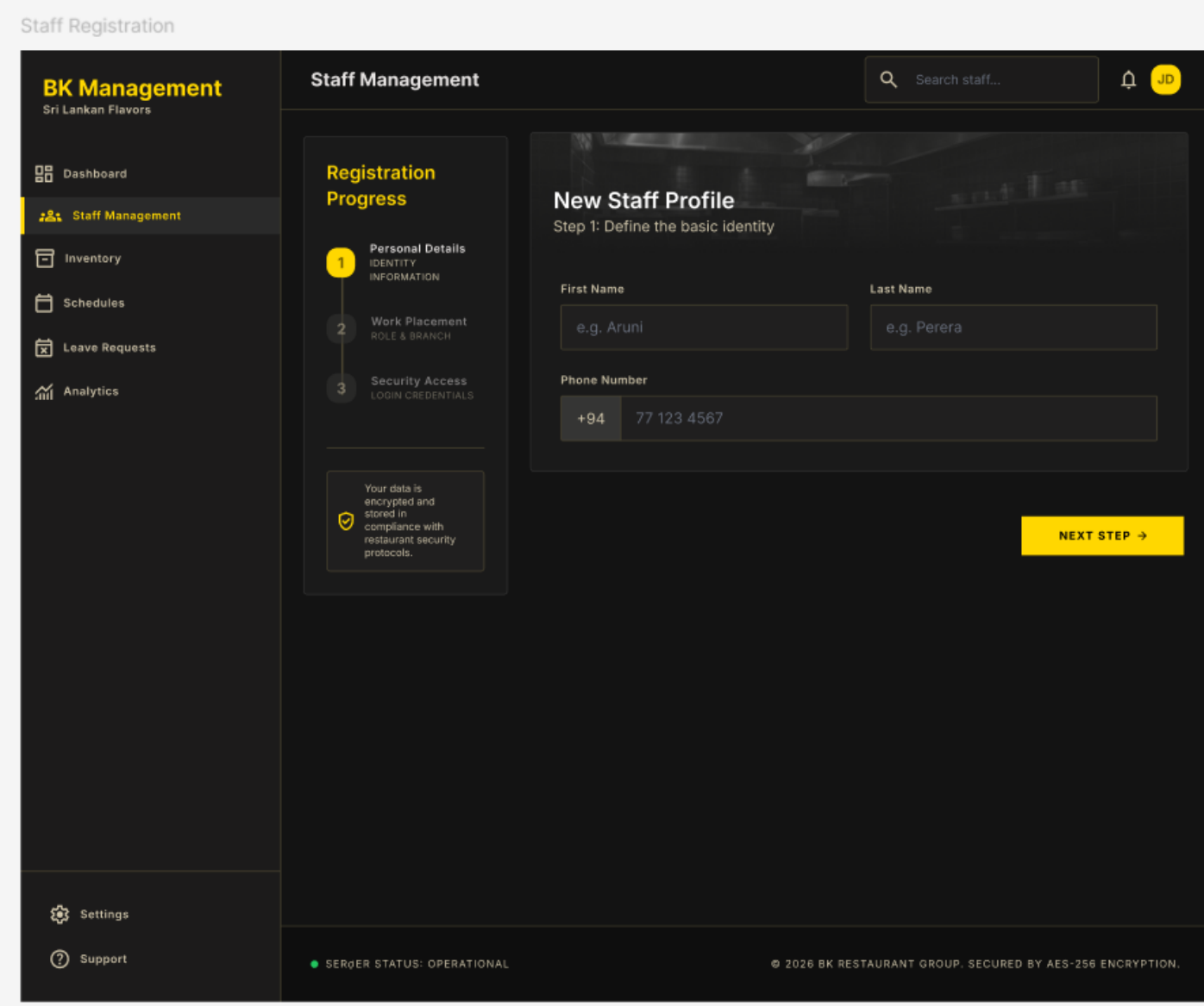
5. UI Design

5.1 Wireframes for All Major Screens

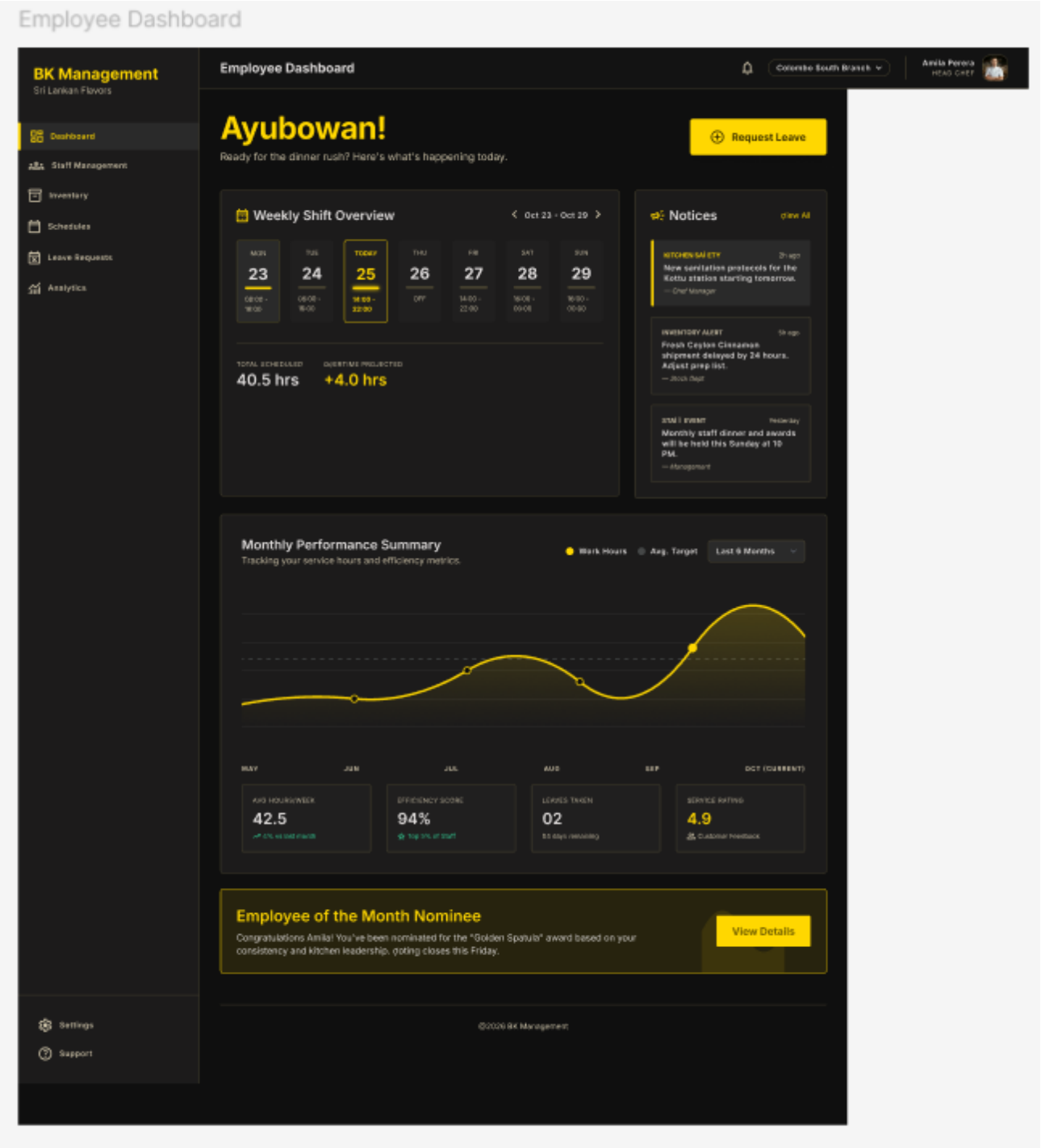
Wireframes are provided for each major screen of the web (POS, KDS, Owner Dashboard) and mobile (Rider App) applications. Each wireframe should reflect the simple, low-training-overhead design described in the SRS feasibility study (Section 6.2.3).

5.1.1 Employee Related Screens

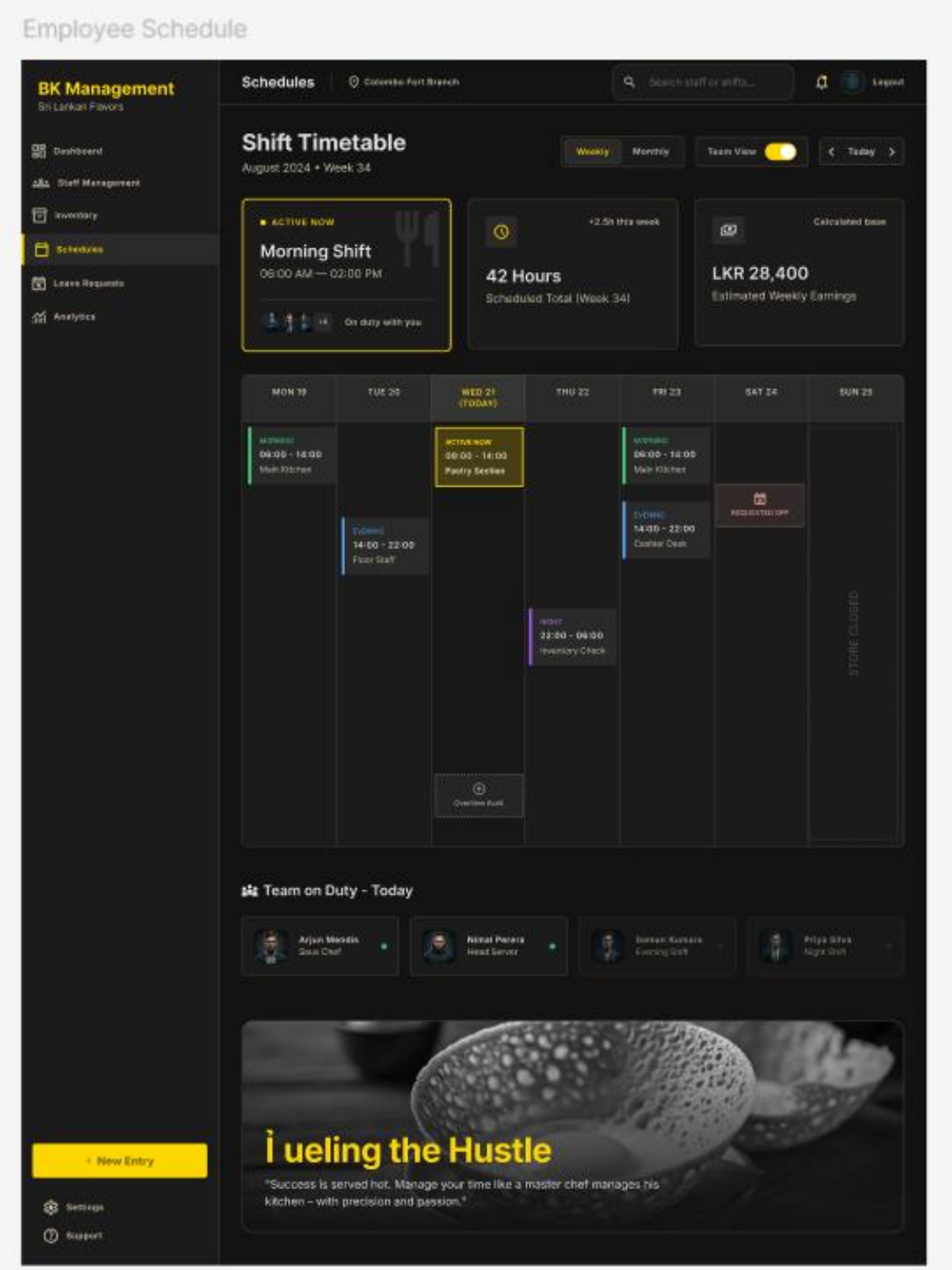
5.1.1.1 Staff Registration



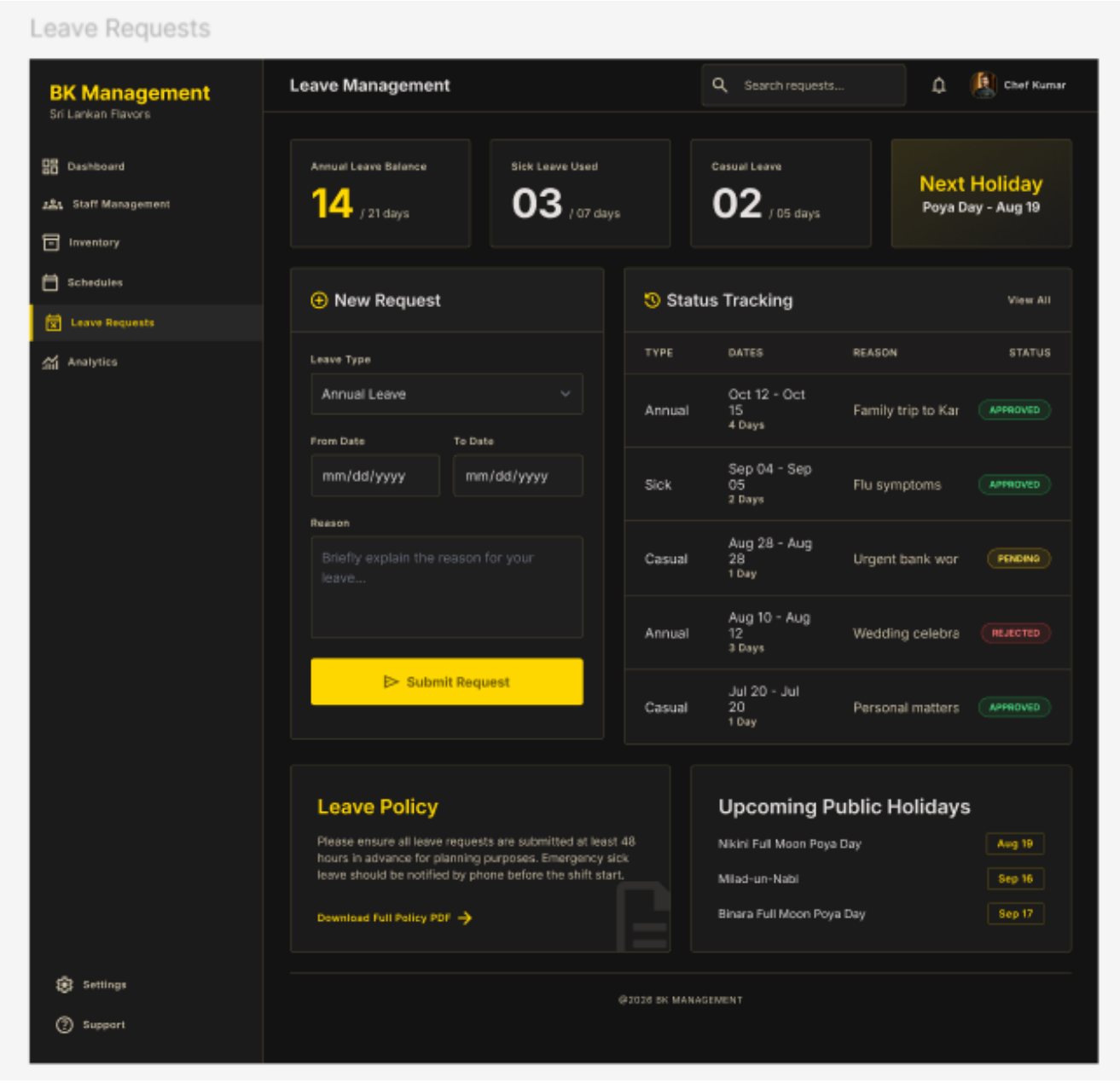
5.1.1.2 Employee Dashboard



5.1.1.3 Employee Schedule

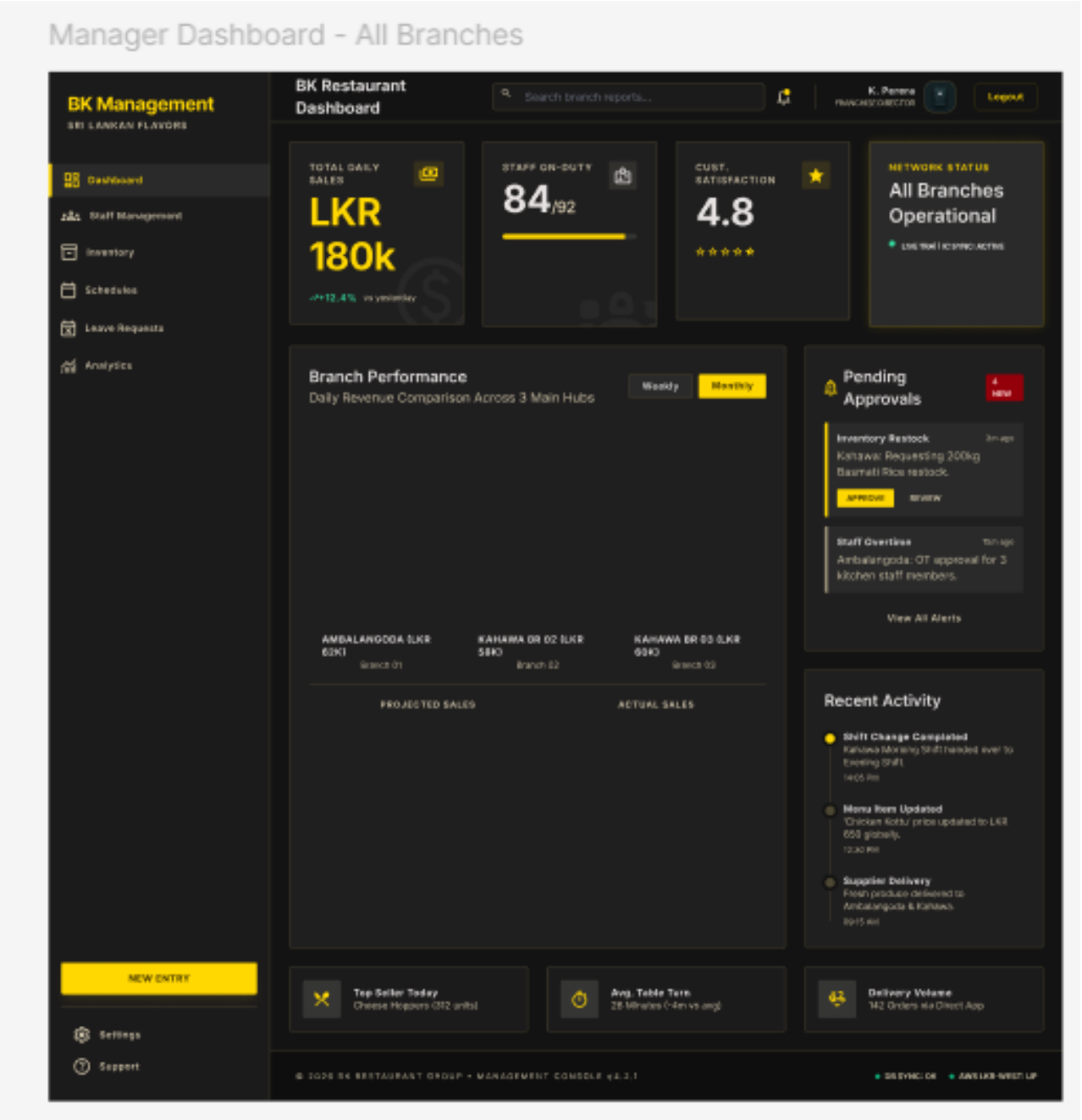


5.1.1.4 Leave Requests

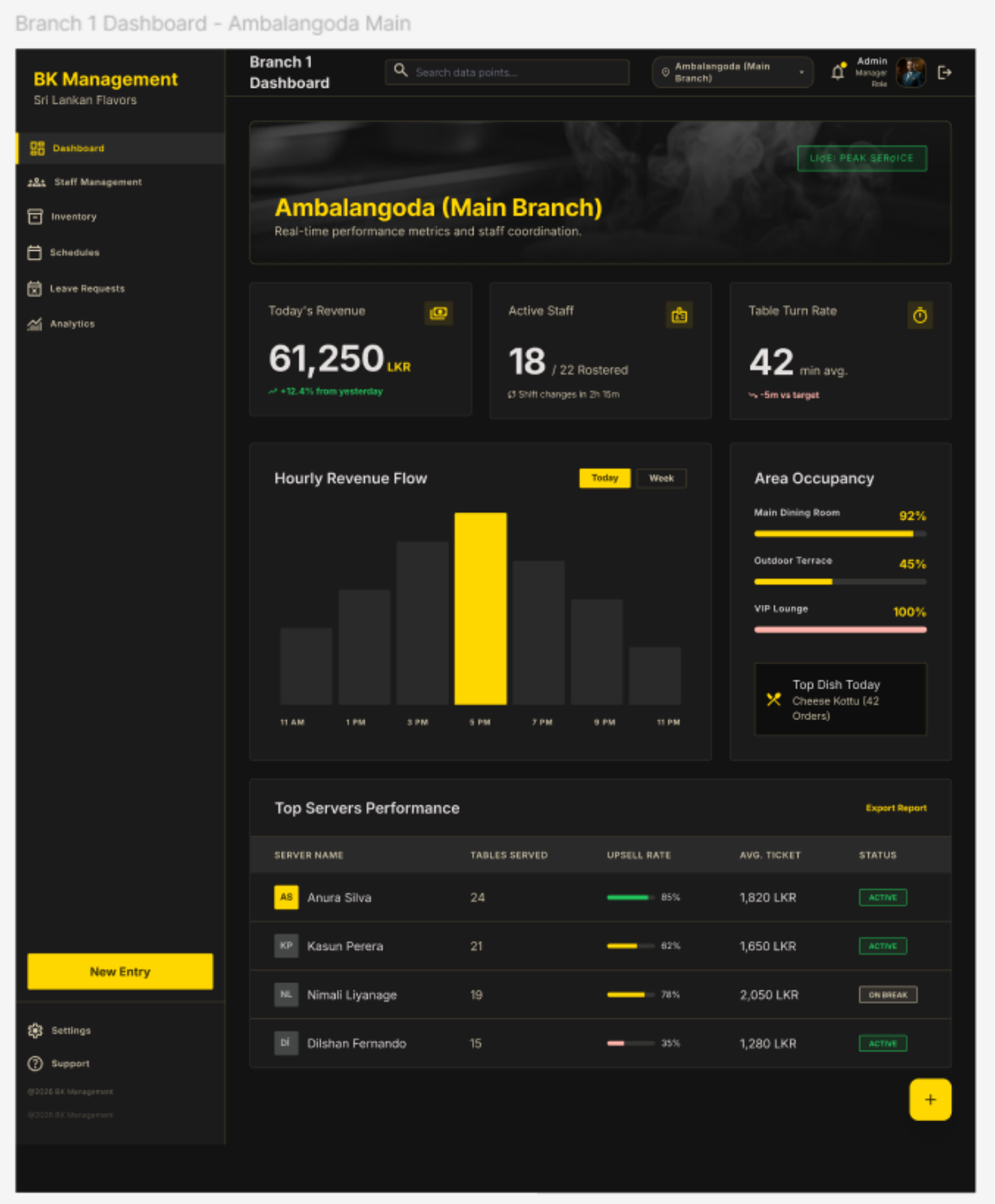


5.1.2 Manager Related Screens

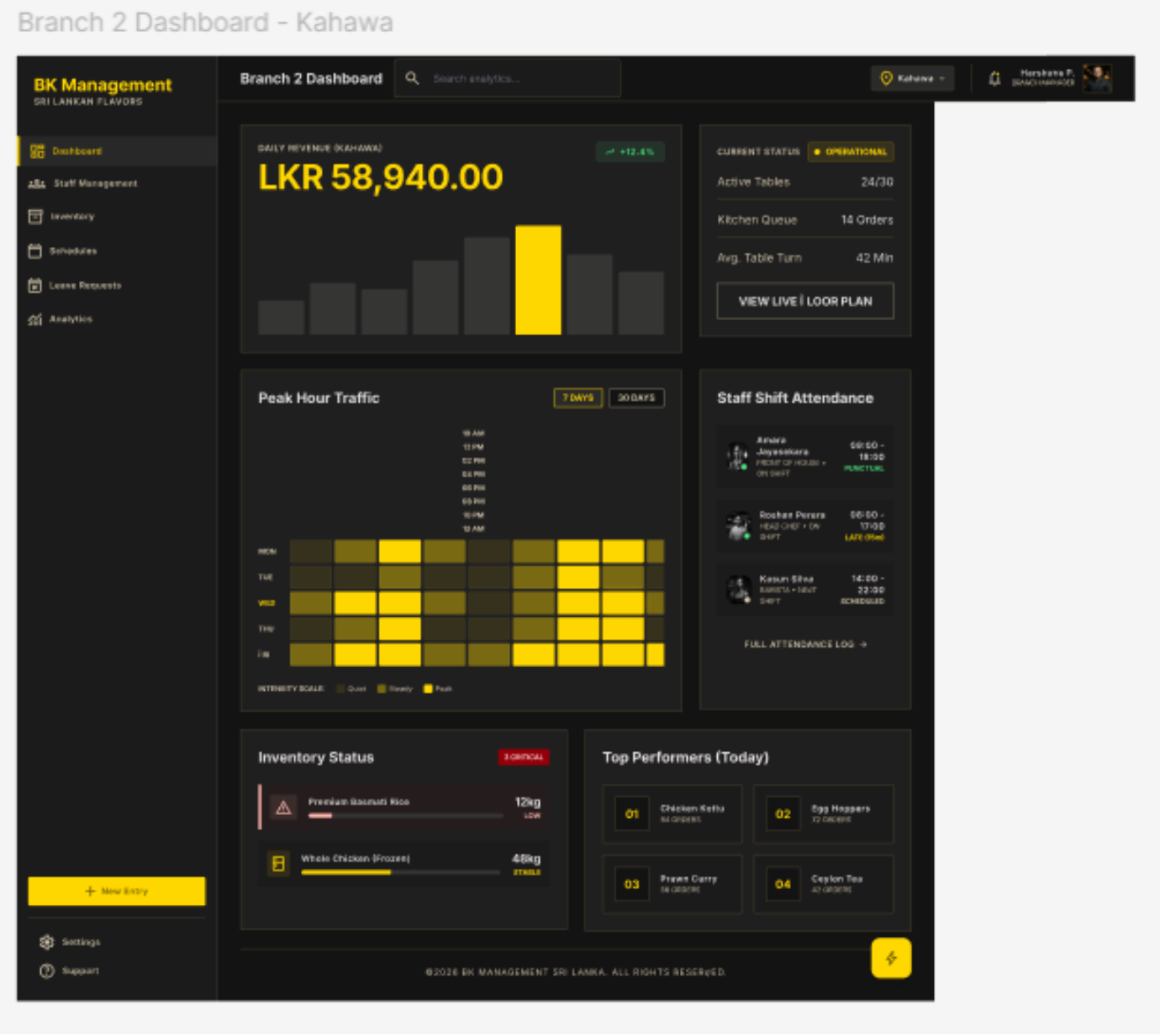
5.1.2.1 Manager Dashboard - All Branches



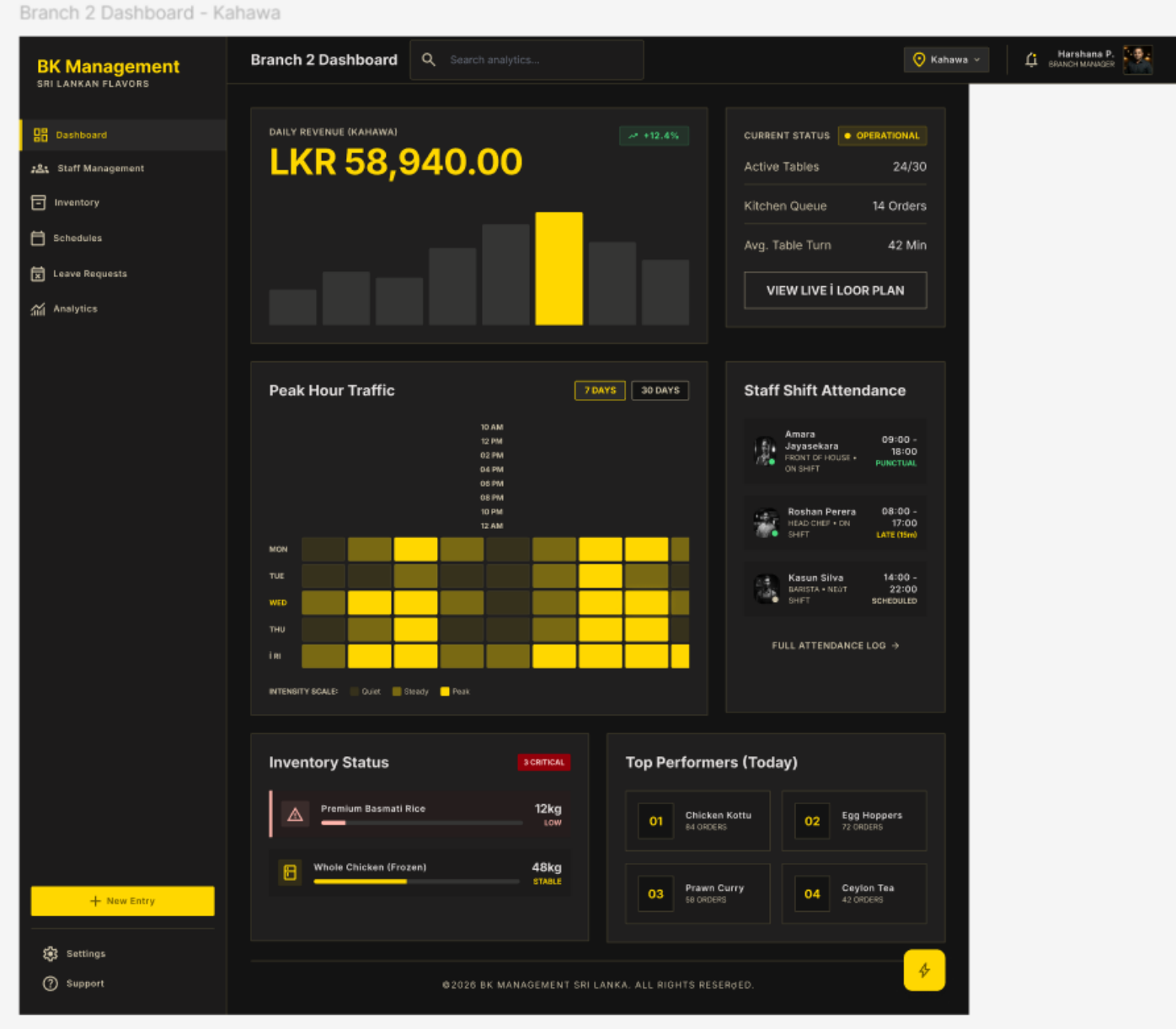
5.1.2.2 Branch 1 Dashboard - Ambalangoda Main



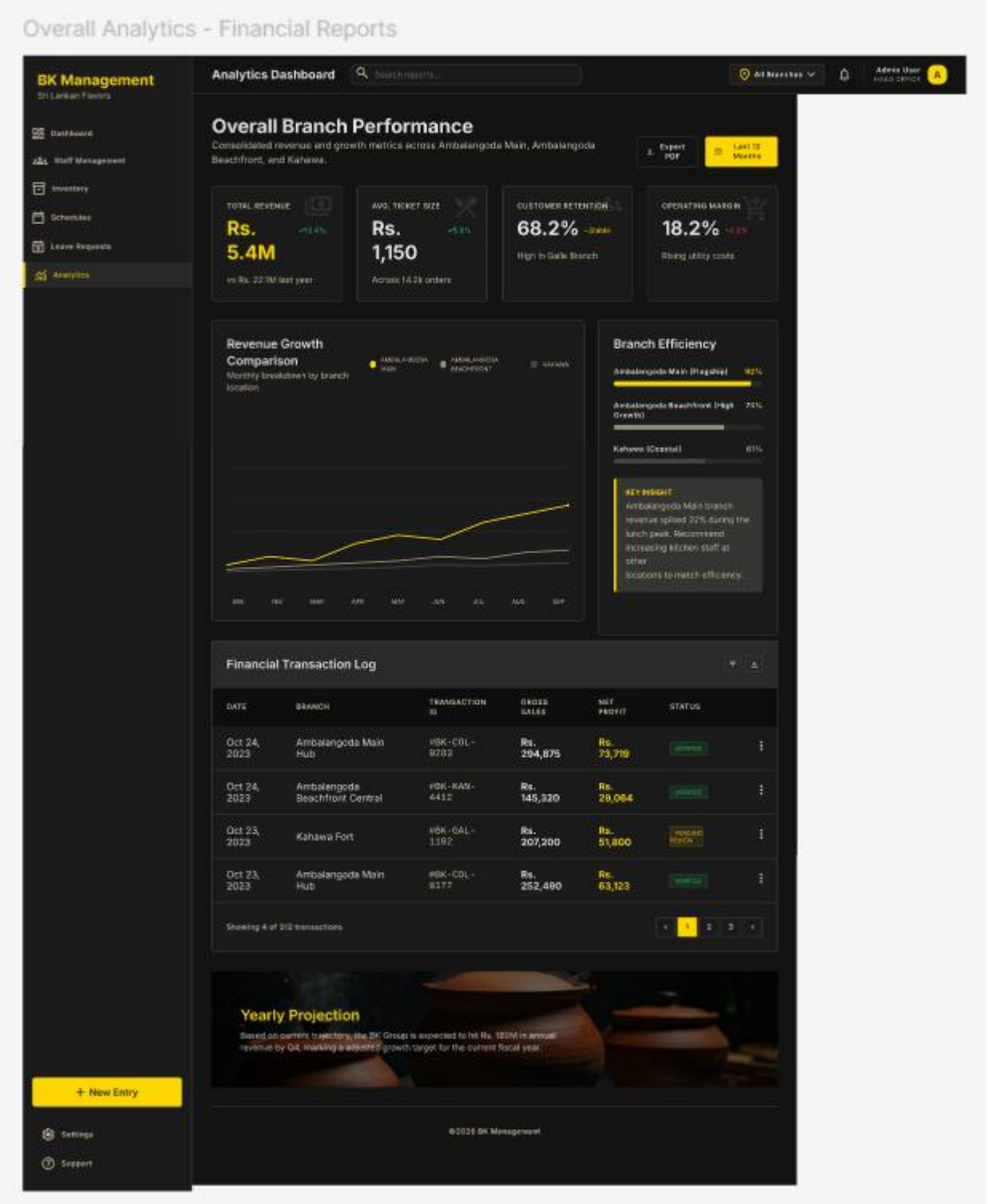
5.1.2.3 Branch 2 Dashboard – Kahawa



5.1.2.4 Branch 3 Dashboard – Kahawa

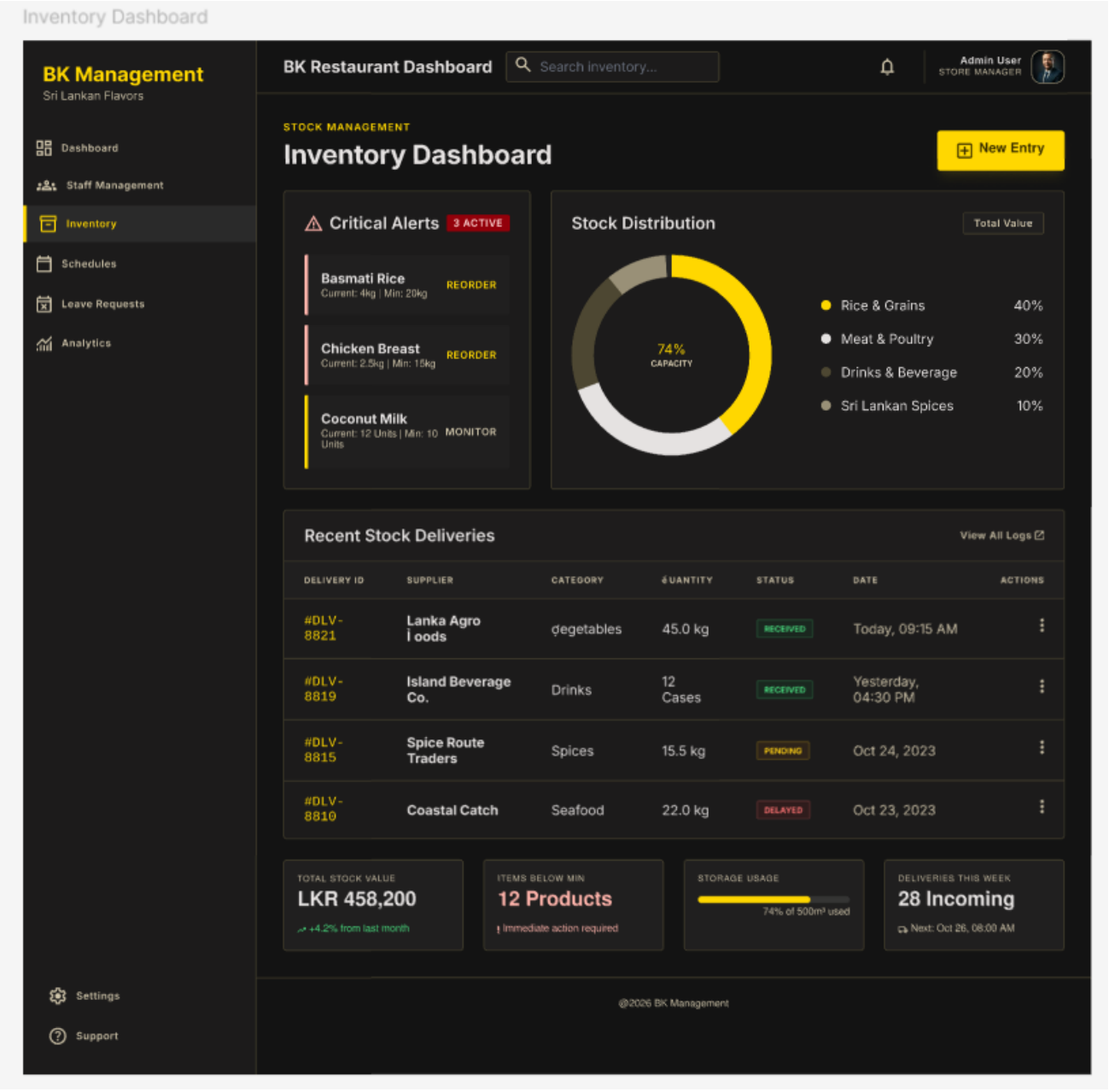


5.1.2.5 Overall Analytics - Financial Reports



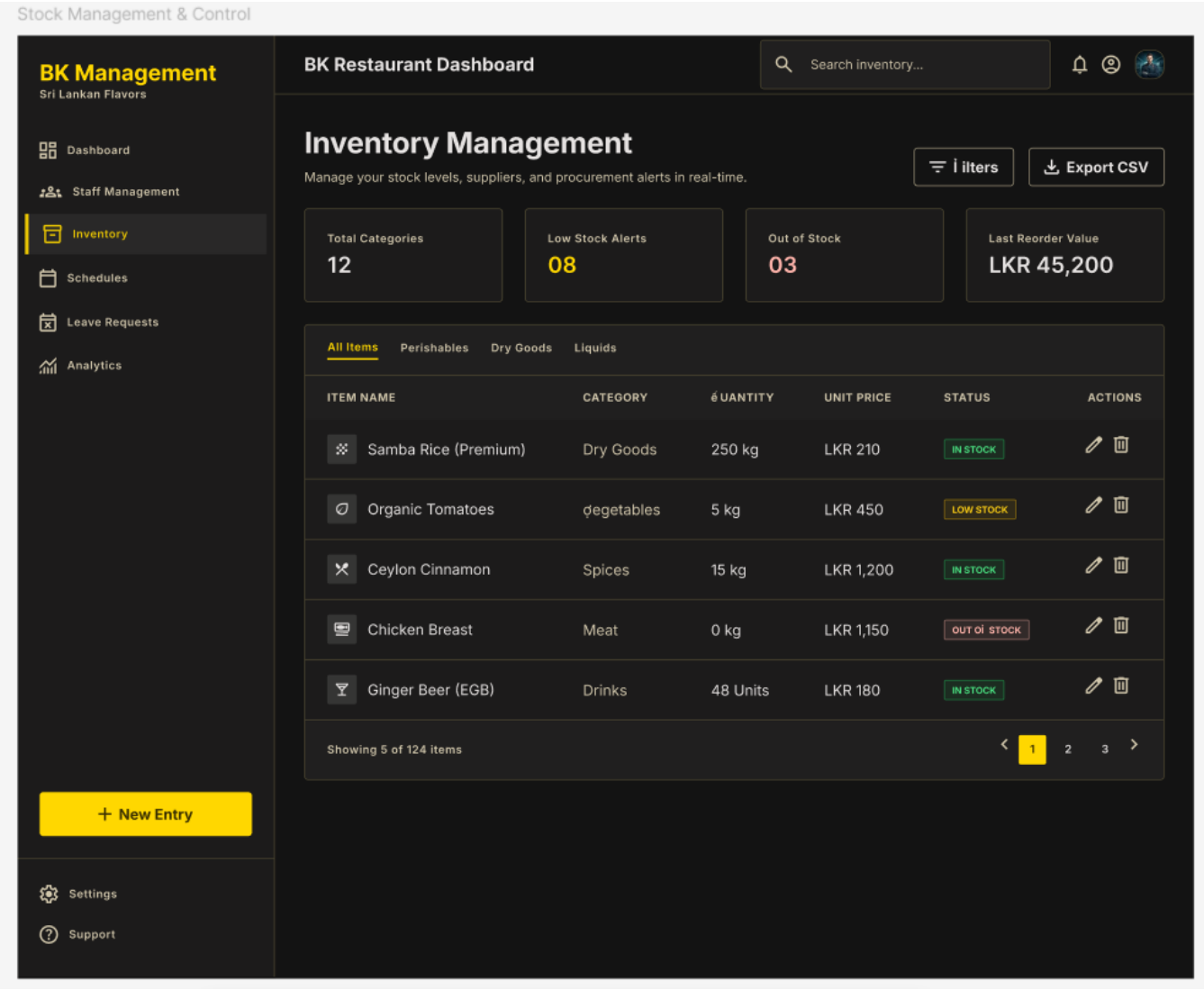
5.1.3 Inventory Related Screens

5.1.3.1 Inventory Dashboard

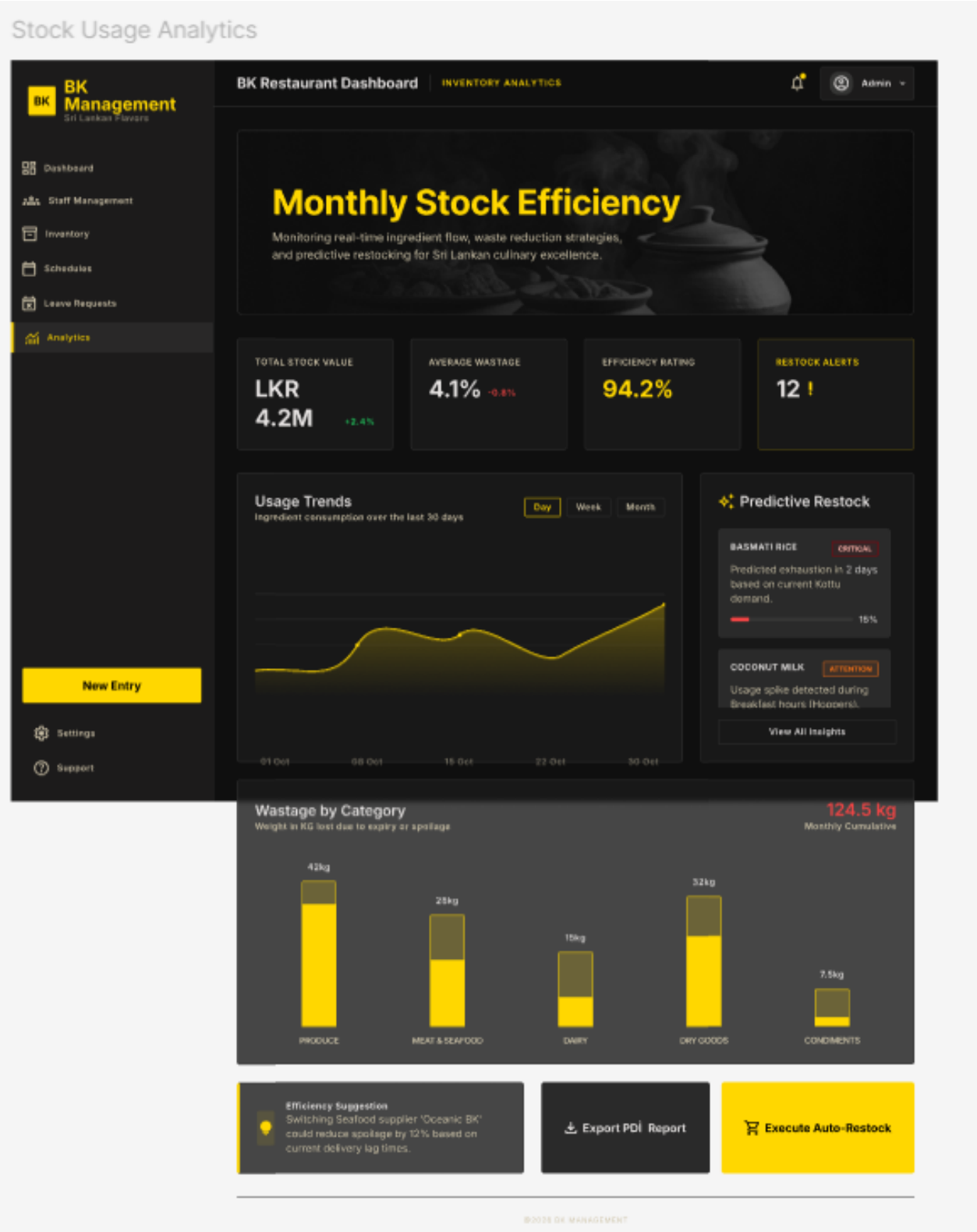


5.1.3.2 Stock Management & Control

Stock Management & Control

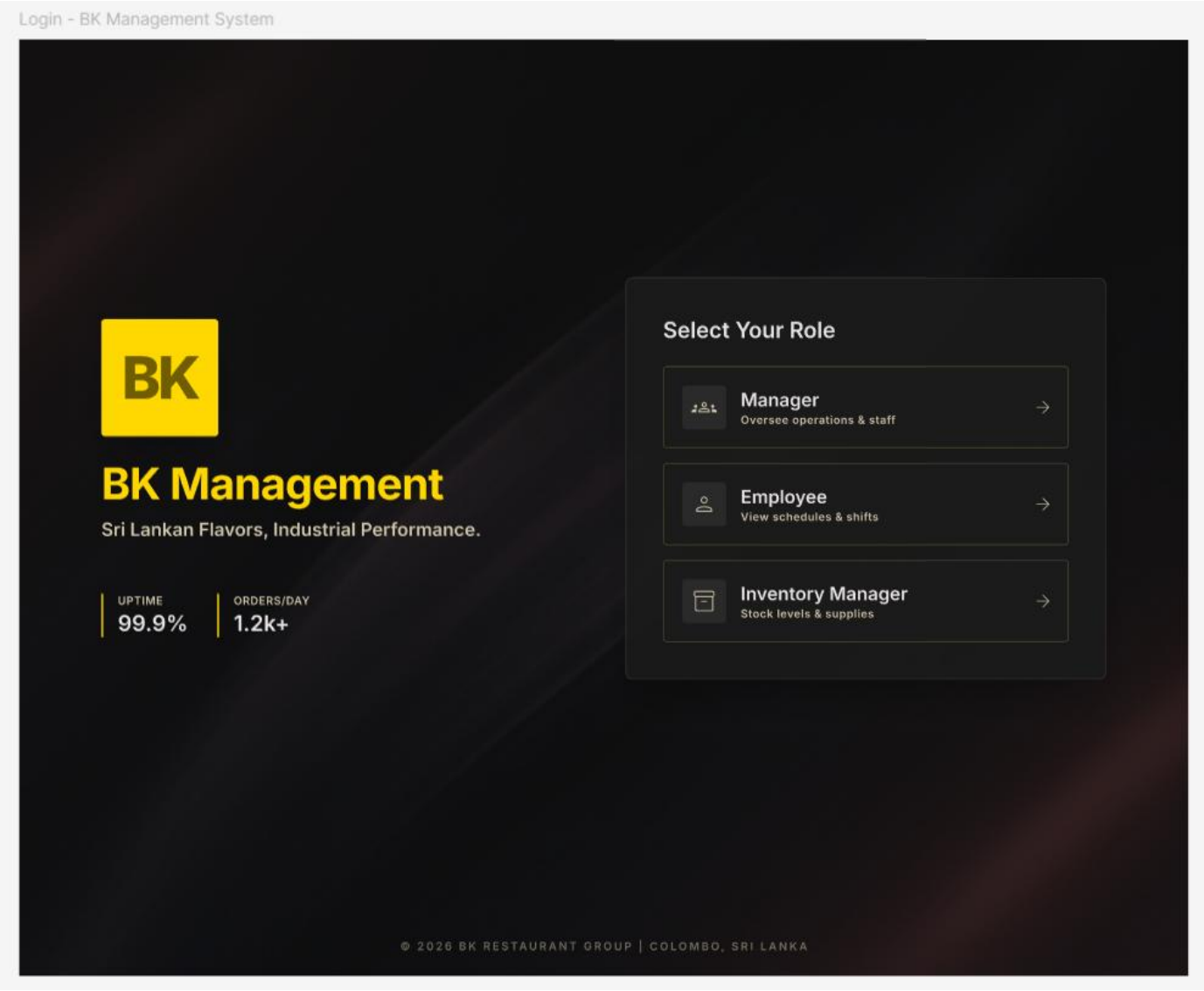


5.1.3.3 Stock Usage Analytics

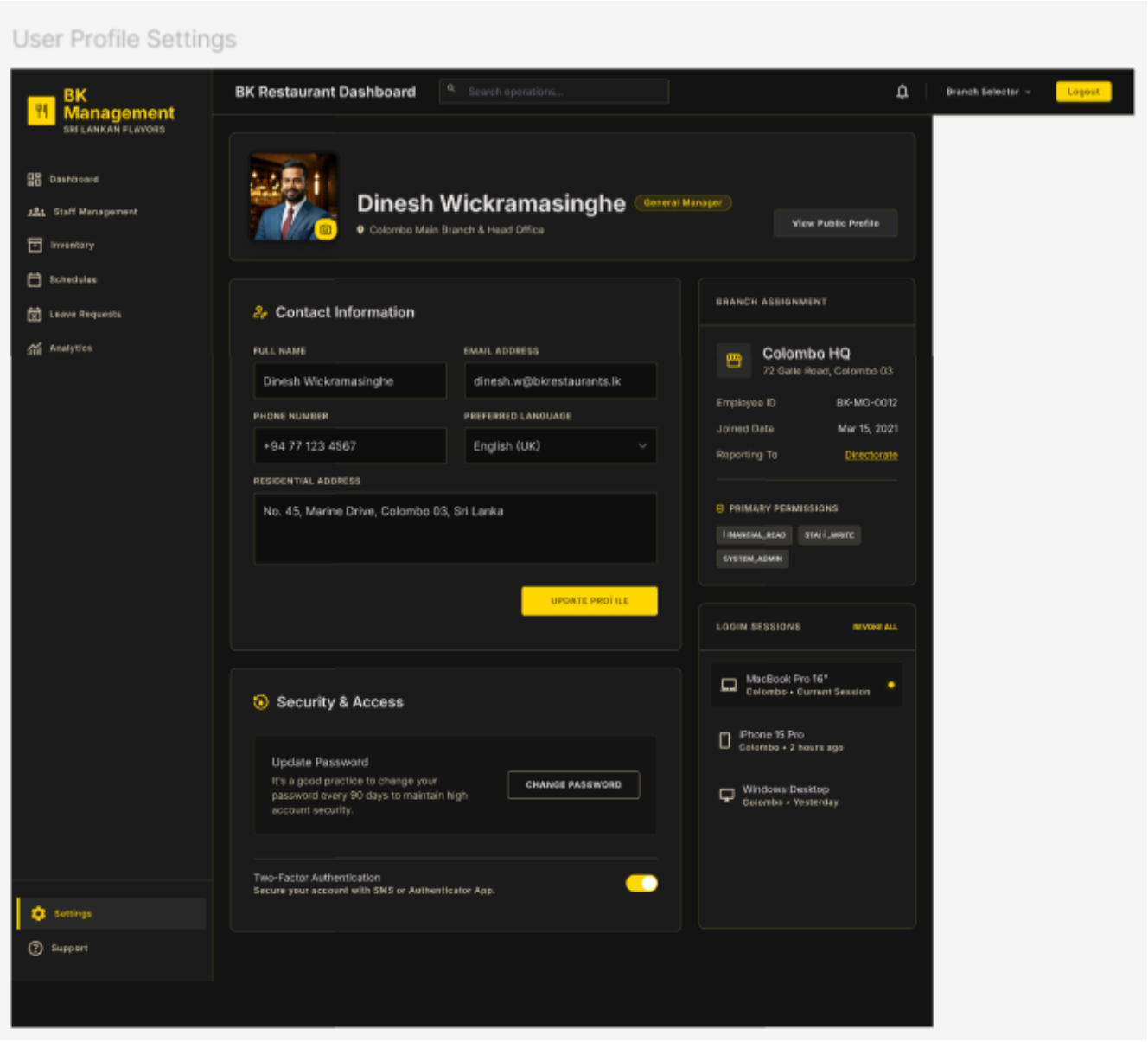


5.1.4 Common Screens

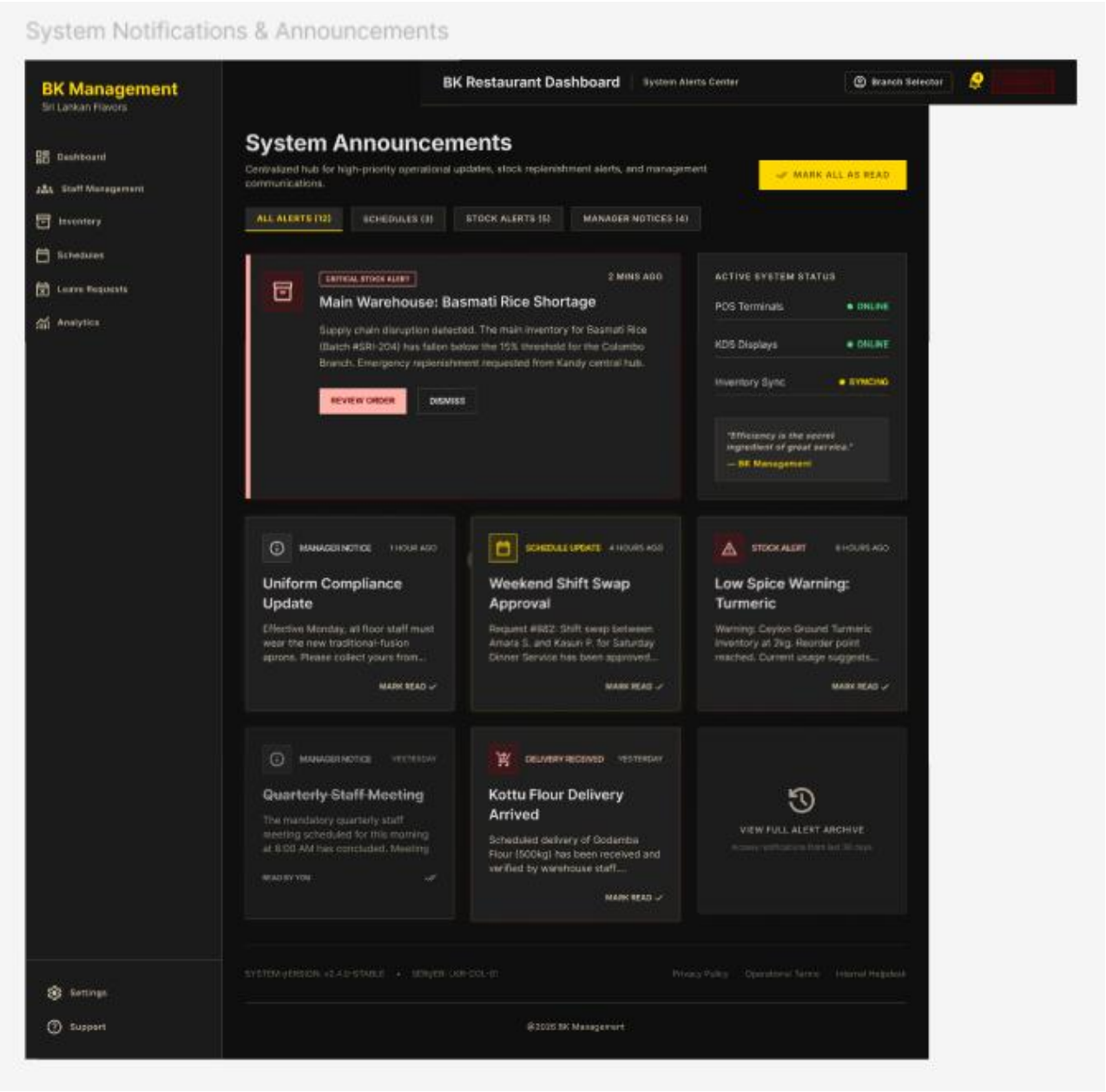
5.1.4.1 Login - BK Management System



5.1.4.2 User Profile Settings



5.1.4.3 System Notifications & Announcements

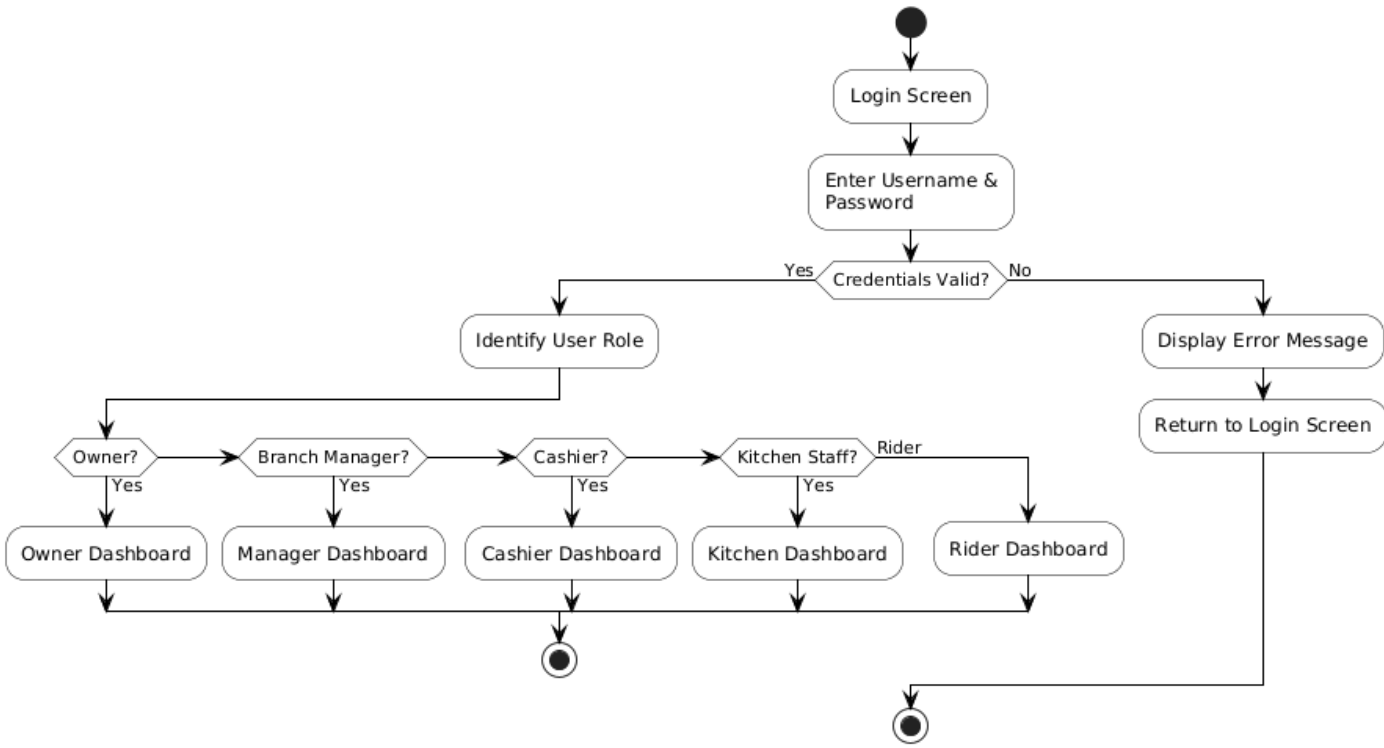


5.2 Navigation Flow Diagram

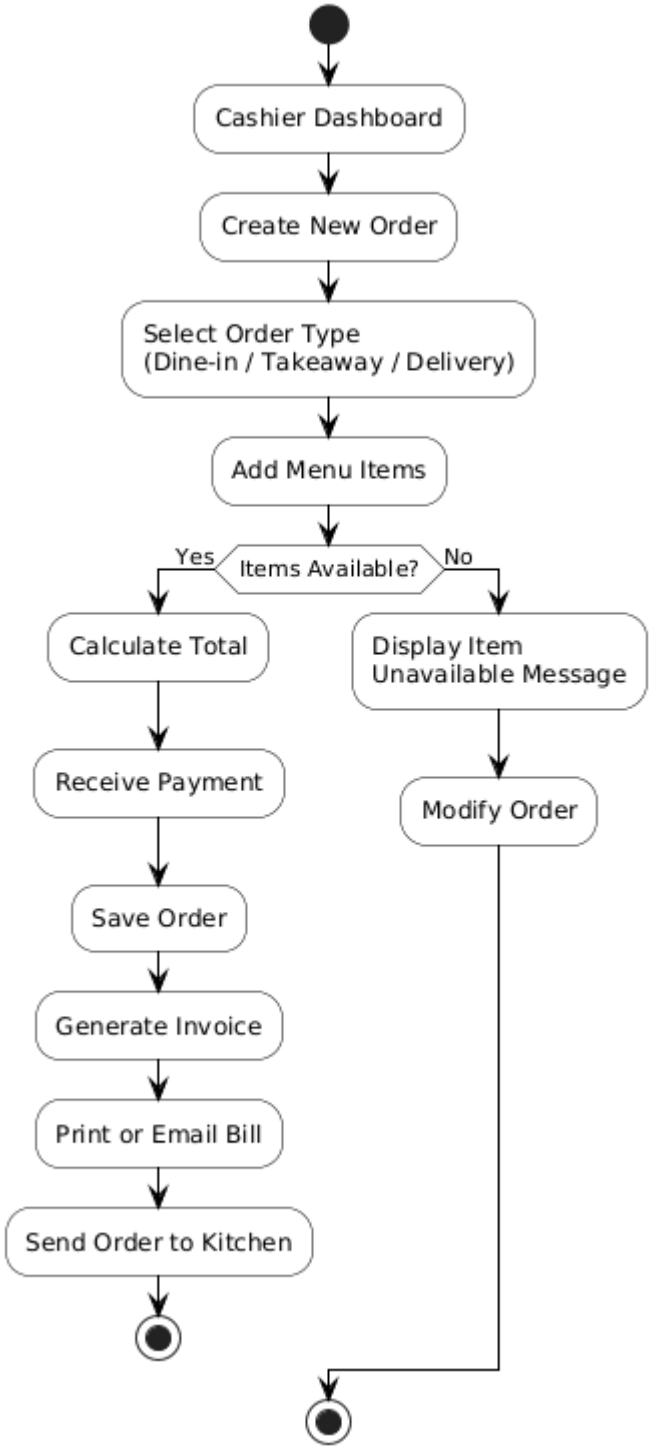
The navigation flow diagram below shows how users move between screens based on their role, from login through to each major feature area.

1. User Login & Role-Based Access Flow
2. Customer Order Creation & Billing Flow
3. Kitchen Order Processing Flow
4. Delivery Rider Flow
5. Inventory Management & Supplier Request Flow
6. Owner Monitoring & Analytics Flow

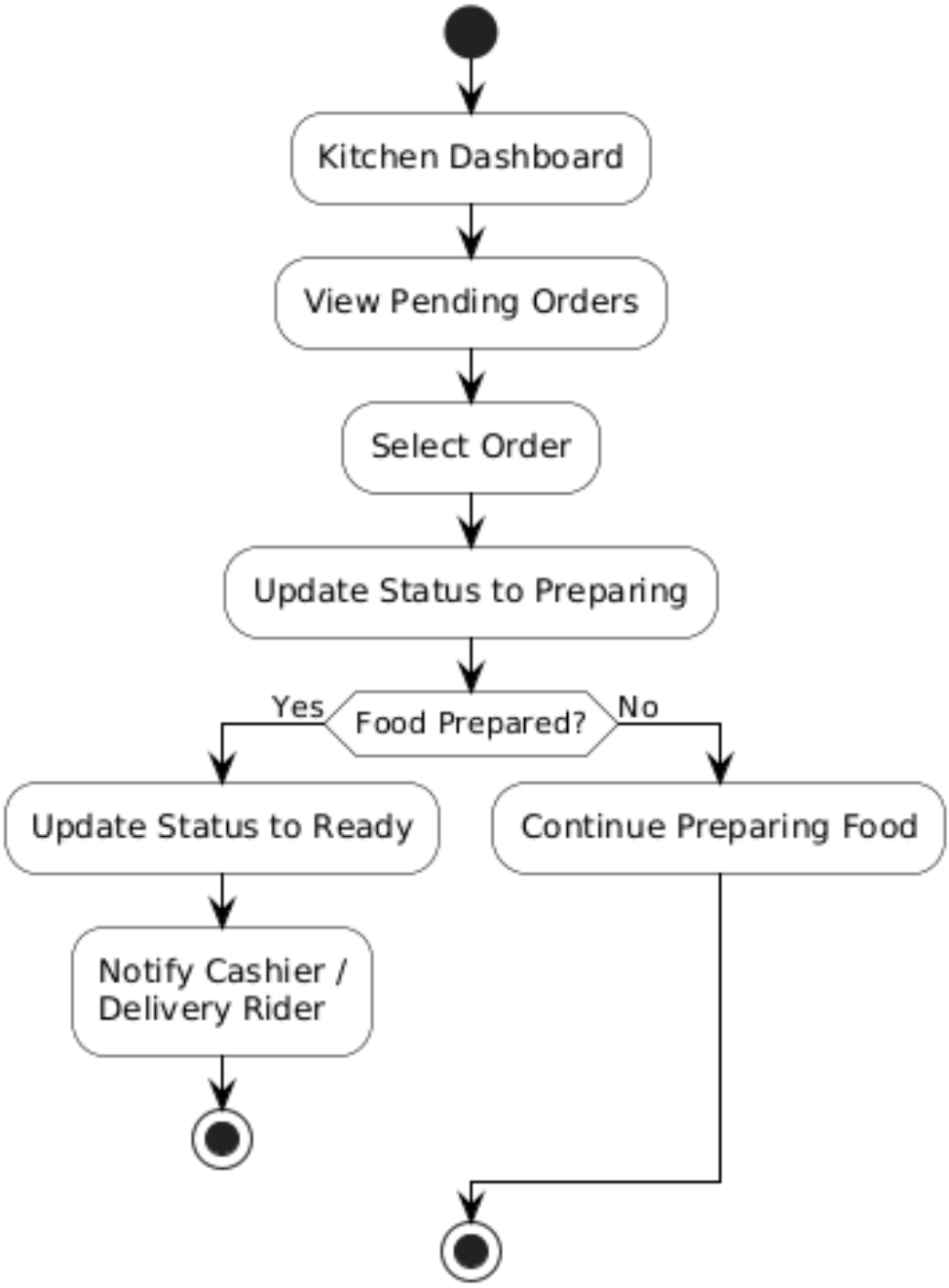
5.2.1 User Login & Role-Based Access Flow



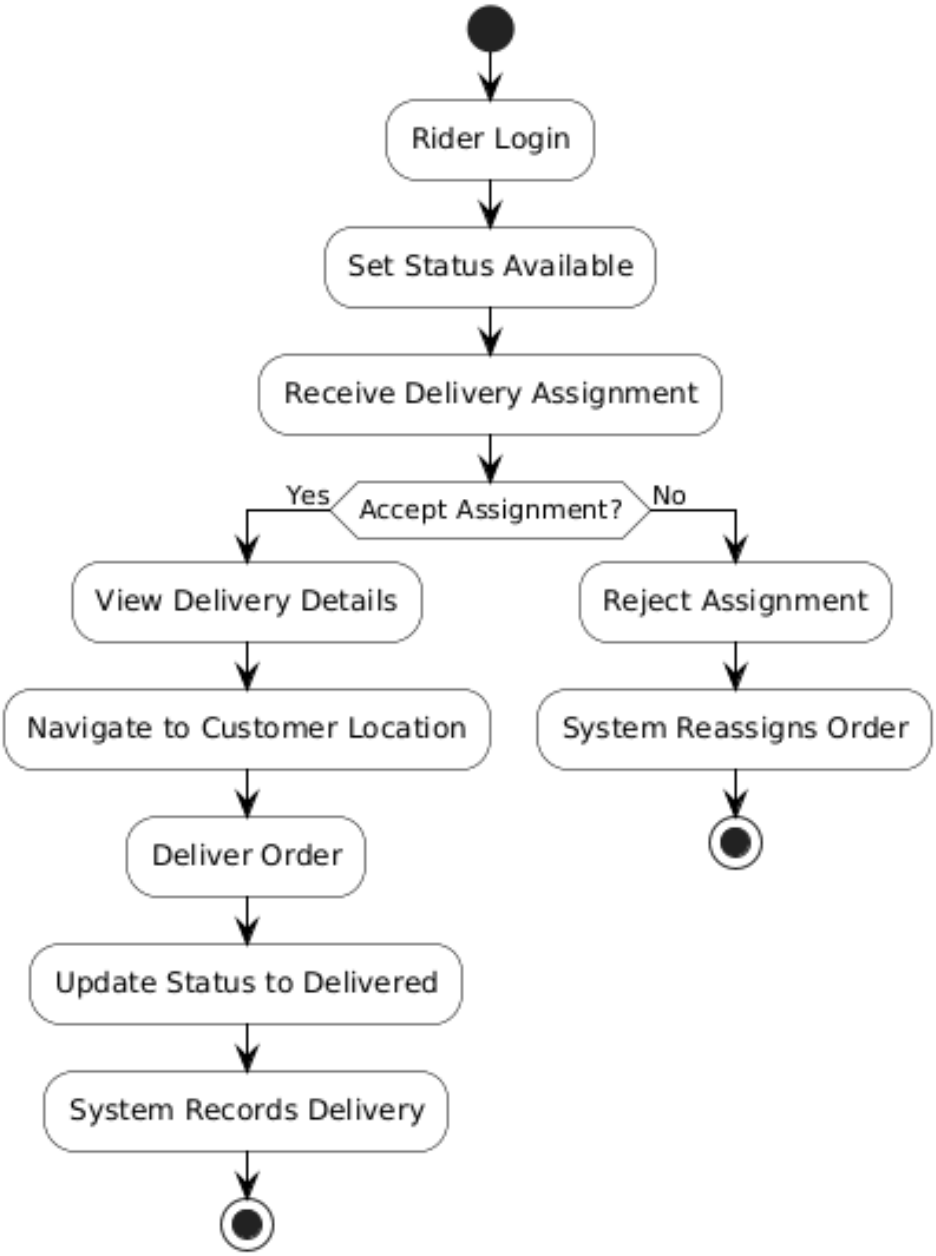
5.2.2 Customer Order Creation & Billing Flow



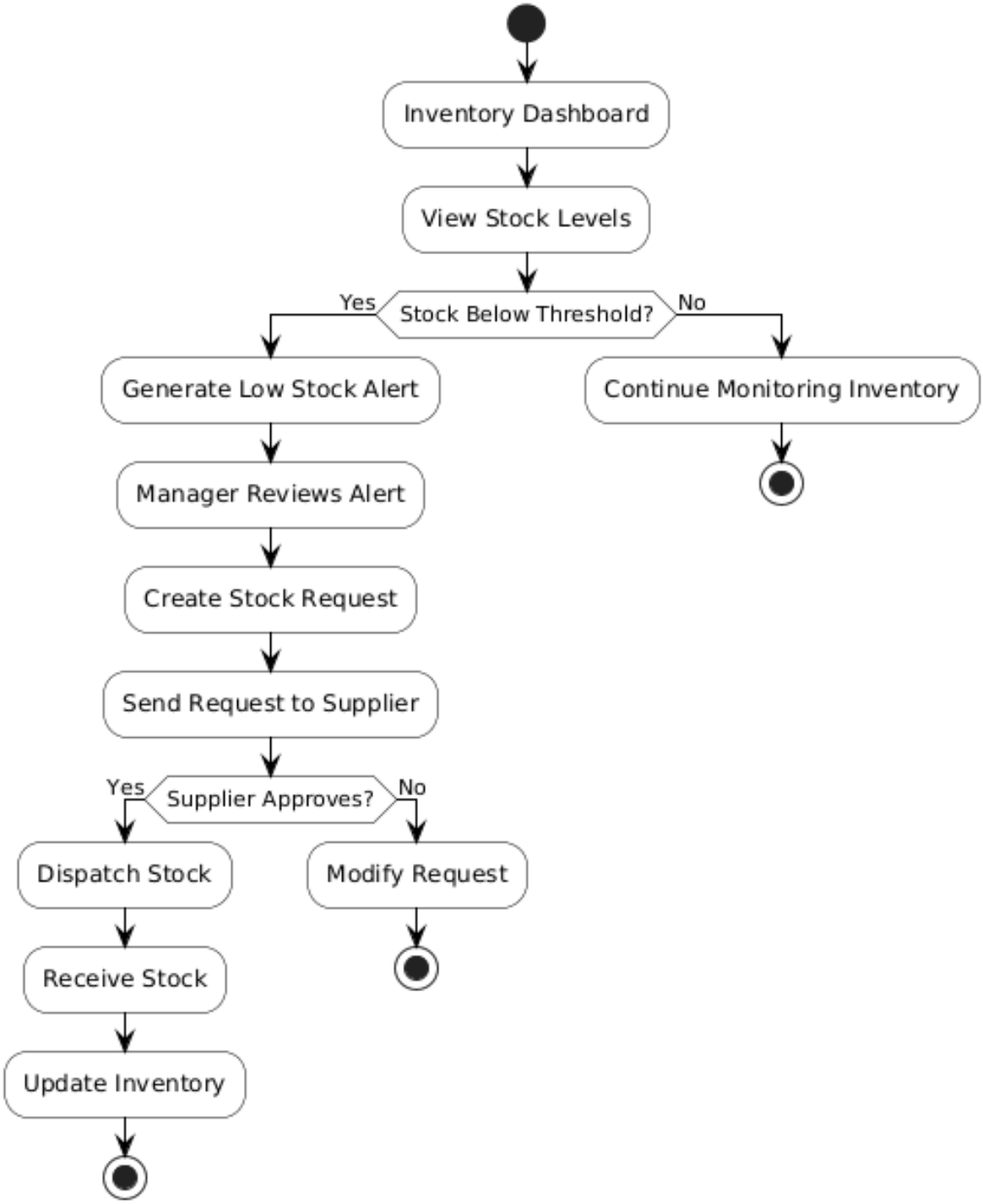
5.2.3 Kitchen Order Processing Flow



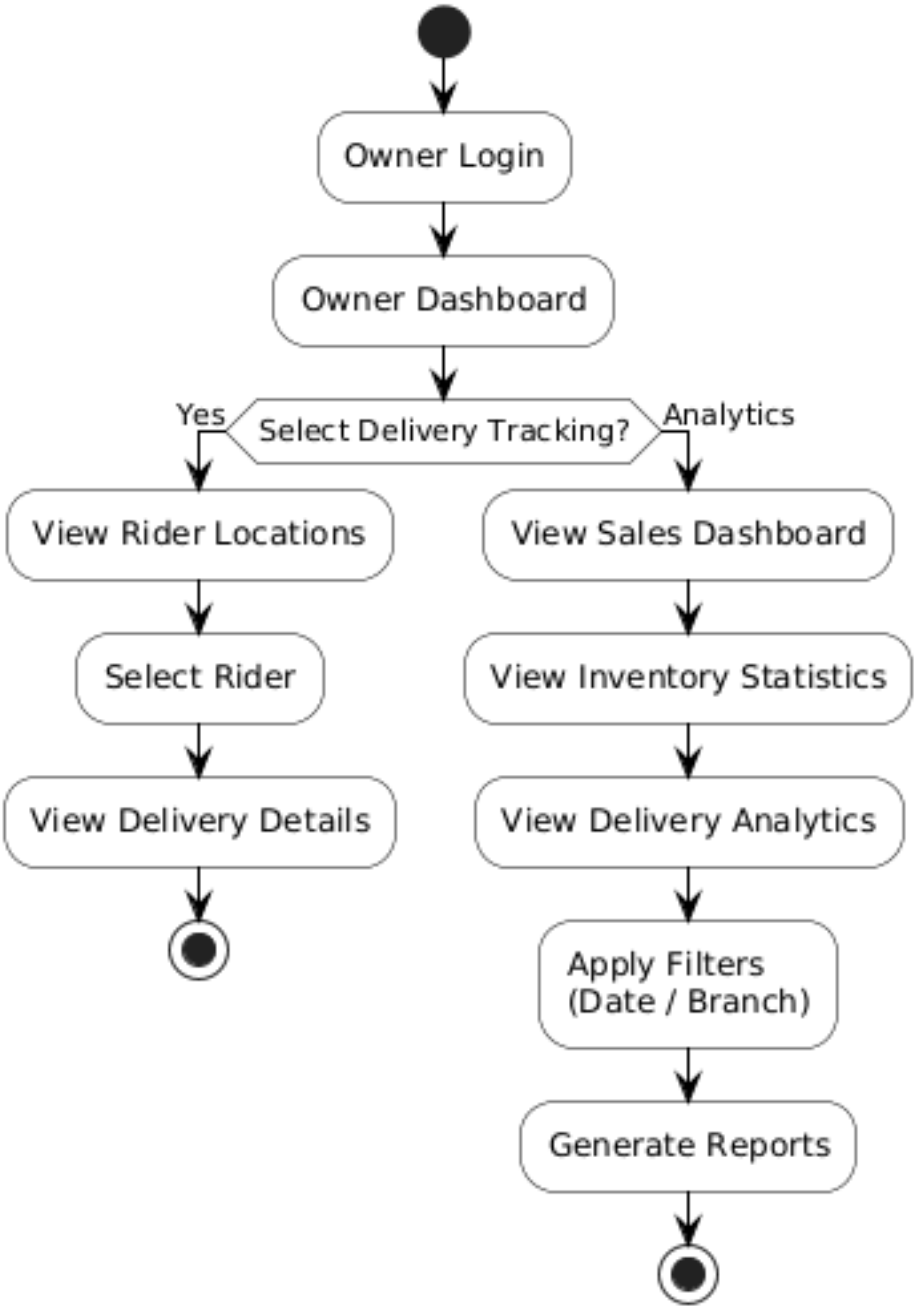
5.2.4 Delivery Rider Flow



5.2.5 Inventory Management & Supplier Request Flow



5.2.6 Owner Monitoring & Analytics Flow



5.3 UX Decisions and Rationale

UX Decision	Rationale
Large, tappable tiles for POS menu items and KDS actions	Reception and kitchen staff have low-to-medium technical proficiency (SRS Section 2.2). Large touch targets reduce input errors and support the 30-minute training target identified in the feasibility study (SRS Section 6.2.3).
Colour-coded status indicators (e.g., green/red for stock levels, order status colours on KDS)	Allows staff to recognise urgent items (low stock, new orders) at a glance without reading detailed text, supporting NFR-08 and NFR-09.
Step-by-step guided flow in the Rider App with integrated navigation	Delivery riders are smartphone-literate but benefit from minimal-decision interfaces; a guided flow with one primary action per screen reduces errors during delivery (NFR-10).
Single-page analytics dashboard with pre-built charts for the Owner	The Owner persona requires high-level visibility without manual data entry; pre-built charts and filters (date range, branch) align with the 'standard web conventions' expectation noted in the SRS feasibility study.
Consistent navigation bar/menu structure across all web screens, scoped to the logged-in user's role	Reinforces role-based access control (FR-17) at the UI level and ensures users are not shown features they cannot use, reducing confusion.
Real-time updates (e.g., new orders on KDS, delivery status on Owner Dashboard) without manual refresh	Matches the real-time processing requirements (NFR-01, NFR-03) and removes the need for staff to repeatedly refresh screens.
Confirmation messages and undo-safe flows for cancellations and stock adjustments	Reduces the risk of accidental data loss for non-technical users and supports data consistency (NFR-12).

6. Interface Design

6.1 External Interfaces (APIs, Hardware, Third-Party Services)

The system integrates with the following external interfaces:

External Interface	Purpose	Integration Notes
Google Maps Platform (Maps SDK, Directions API, Geocoding API)	Provides live GPS tracking for delivery riders, route navigation, and delivery zone validation (FR-07, UC-09).	Accessed via REST/SDK calls from the Delivery Module (backend) and the Rider App and Owner Dashboard (frontend). Requires a Google Maps API key, with usage restricted by domain/app for security.
Thermal Receipt Printer	Prints physical bills/invoices at the POS counter (UC-05).	Connected to POS terminals via USB or network printing; the Orders Module sends a formatted print job (e.g., via a printer driver/SDK or a local print server) when 'Print Invoice' is selected.
Email/SMS Notification Gateway (e.g., SendGrid for email, Twilio for SMS)	Sends digital invoices to customers, notifies suppliers of stock requests, and notifies employees of schedules (FR-05, UC-05, UC-12, UC-13).	Accessed via REST API from a shared Notification Service used by multiple backend modules; API keys stored securely as environment variables (see Section 7.2).
Cloud Hosting Platform (AWS / Railway.app)	Hosts the Node.js/Express API, PostgreSQL database, and Redis cache (SRS Section 2.3, 6.2.2).	The API and database are deployed as managed services; configuration (connection strings, secrets) is provided via environment variables, not hard-coded.
Modern Web Browsers (Chrome, Edge, Firefox, Safari)	Client runtime for the React-based POS, KDS, and Owner Dashboard (NFR-19).	The frontend uses standard, widely-supported web APIs and avoids browser-specific features to ensure cross-browser compatibility.
Android OS / GPS Hardware	Runtime environment and location sensor for the Flutter Rider App (NFR-20).	The app requests location permissions at runtime and uses background location updates while a delivery is in progress.

6.2 Internal Module Interfaces

Internal communication between frontend clients and the backend, and between backend modules, takes place over a versioned REST API (e.g., /api/v1/...) using JSON payloads and JWT-based authentication headers. The table below summarises the primary internal interfaces exposed by each backend module.

Module	Representative Endpoints	Consumed By
Authentication Module	POST /api/v1/auth/login, POST /api/v1/auth/refresh, GET /api/v1/auth/me	Web Client (POS, KDS, Owner Dashboard), Mobile Client (Rider App)
Orders Module	POST /api/v1/orders, GET /api/v1/orders/{id}, PATCH /api/v1/orders/{id}, PATCH /api/v1/orders/{id}/status, POST /api/v1/orders/{id}/finalize	POS UI, KDS UI
Inventory Module	GET /api/v1/inventory, PATCH /api/v1/inventory/{id}, GET /api/v1/inventory/alerts, POST /api/v1/stock-requests, PATCH /api/v1/stock-requests/{id}	Inventory UI (Branch Manager), Orders Module (internal call to check/deduct stock)
Delivery Module	POST /api/v1/deliveries, PATCH /api/v1/deliveries/{id}/status, GET /api/v1/deliveries/active, GET /api/v1/deliveries/{id}/location	Rider App, Owner Dashboard, Orders Module (internal call on order ready)
Employee Module	POST /api/v1/shifts, GET /api/v1/shifts, POST /api/v1/attendance, POST /api/v1/leave-requests, PATCH /api/v1/leave-requests/{id}	Employee Scheduling UI, Attendance UI
Reporting Module	GET /api/v1/analytics/sales, GET /api/v1/analytics/inventory, GET /api/v1/analytics/delivery	Owner Dashboard

Internal module-to-module calls (e.g., the Orders Module calling the Inventory Module to check stock availability, or the Orders Module calling the Delivery Module when an order becomes 'Ready for Delivery') are implemented as direct service-layer function calls within the same Node.js process for simplicity, with the option to extract them as internal HTTP/message-queue calls if the modules are later split into separate services to support NFR-14 and NFR-15.

7. Security Design

7.1 Authentication and Authorisation Strategy

The system uses a centralised Authentication Module to manage identity and access across all clients:

- Users authenticate with an email/username and password (UC-01, FR-18). Passwords are hashed using a strong, salted hashing algorithm (e.g., bcrypt) before storage, addressing NFR-05.
- On successful login, the Authentication Module issues a signed JSON Web Token (JWT) containing the user's ID, role, and branch ID. The token has a limited expiry, with a refresh-token mechanism to obtain new access tokens without re-entering credentials.
- Every protected API endpoint validates the JWT and extracts the user's role and branch ID before processing the request, addressing FR-18 and NFR-04.
- Role-Based Access Control (RBAC) is enforced at the API layer using middleware that checks the user's role against the permissions required for each endpoint (e.g., only Branch Managers and Restaurant Owners can access inventory adjustment endpoints), addressing FR-17 and NFR-06.
- Branch-level data isolation is enforced by filtering queries on branchId for roles other than Restaurant Owner, ensuring Branch Managers and staff can only access data for their own branch.
- All significant actions (order modifications/cancellations, inventory adjustments, stock request approvals, schedule changes) are recorded in an audit log table with user ID, action, timestamp, and affected entity, addressing NFR-07.

7.2 Data Protection (At Rest, In Transit)

- In Transit: all communication between clients (web, mobile) and the API occurs over HTTPS/TLS. The API does not accept plain HTTP connections in production, protecting login credentials, customer data, and order details from interception (NFR-04).
- At Rest: passwords are stored as salted bcrypt hashes, never in plain text. Sensitive configuration values (database credentials, JWT signing keys, third-party API keys) are stored as environment variables/secrets, not committed to source control.
- Database Access Control: the database is not exposed directly to the public internet; only the backend API server can connect to it, using a dedicated database user with the minimum privileges required.
- Backups: scheduled automated backups of the PostgreSQL database are taken (e.g., daily) and stored securely, with a documented restore procedure, addressing NFR-13.
- Personal Data: customer contact details (for delivery orders) and employee personal data (attendance, leave) are only accessible to roles that require them for their job function, consistent with the RBAC model in Section 7.1.

7.3 Input Validation Strategy

- All API endpoints validate incoming request bodies against a defined schema (e.g., using a validation library such as Joi or class-validator) before any business logic executes, rejecting malformed or unexpected fields.

- Numeric fields (quantities, prices, totals) are validated to be non-negative and within sensible bounds; the Orders Module enforces that order totals must be greater than zero (per the Business Rule in UC-02).
- Status transition fields (Order.status, Delivery.status, StockRequest.status) are validated against the allowed state machine transitions defined in Section 4.3, preventing invalid transitions (e.g., directly from 'Pending' to 'Completed').
- All database queries use parameterised queries or an ORM's query builder, never string concatenation, to prevent SQL injection.
- File and text inputs (e.g., adjustment reasons, leave request reasons) are length-limited and sanitised before storage and before being rendered in the UI to prevent stored cross-site scripting (XSS).
- GPS coordinates submitted by the Rider App are validated to be within plausible ranges and are cross-checked against the delivery address before a delivery can be marked 'Delivered' (per the Business Rule in UC-08).

7.4 OWASP Top 10 Considerations

OWASP Top 10 Category	Mitigation in this Design
A01: Broken Access Control	Centralised RBAC middleware (Section 7.1) checks role and branch ownership on every request; resource ownership is verified before allowing updates (e.g., a Branch Manager cannot modify another branch's inventory).
A02: Cryptographic Failures	Passwords hashed with bcrypt; all traffic over HTTPS/TLS; JWT signed with a strong secret/key stored outside source control (Section 7.2).
A03: Injection	Parameterised queries/ORM usage and schema-based input validation on all endpoints (Section 7.3) prevent SQL injection and similar attacks.
A04: Insecure Design	Status transitions for Order and Delivery follow explicit state machines (Section 4.3) so invalid business states cannot be reached via crafted requests.
A05: Security Misconfiguration	Environment-based configuration for secrets and database connections; production deployments disable debug endpoints and verbose error messages.
A06: Vulnerable and Outdated Components	Dependencies (React, Express, Flutter packages) are kept up to date and monitored using standard tooling (e.g., npm audit, Dependabot) as part of maintenance (NFR-17, NFR-18).
A07: Identification and Authentication Failures	JWT-based authentication with expiring tokens and refresh tokens; account lockout/rate-limiting can be applied to the login endpoint to mitigate brute-force attempts.
A08: Software and Data Integrity Failures	Audit logging (Section 7.1) records changes to critical records (orders, inventory, stock requests) to detect unauthorised data changes.

A09: Security Logging and Monitoring Failures	Audit logs and application logs capture authentication attempts, authorisation failures, and key business events, supporting later monitoring and incident investigation.
A10: Server-Side Request Forgery (SSRF)	External calls (Maps API, notification gateway) use fixed, configuration-defined endpoints rather than user-supplied URLs, preventing SSRF via untrusted input.

8. Non-Functional Design Decisions

This section maps each Non-Functional Requirement (NFR) from the SRS (Section 5) to the specific design decision in this SDS that addresses it.

(i) Performance		
NFR ID	**Category**	**Design Decision Addressing the NFR**
NFR-01	Response Time	Stateless API layer (Section 3.1) with Redis caching for menu and dashboard data (Section 3.5) keeps POS order update response times within the 2-second target.
NFR-02	Concurrent Access	Horizontally scalable, stateless Express API behind a load balancer, with a single shared PostgreSQL database and connection pooling, supports concurrent access from all branches.
NFR-03	Real-Time Processing	Real-time event notifications (e.g., WebSocket or push-based updates) are used for order status changes (SD-04), delivery status changes (SD-08), and inventory updates (SD-10/SD-11).
(ii) Security		
NFR ID	**Category**	**Design Decision Addressing the NFR**
NFR-04	User Authentication	JWT-based authentication over HTTPS for all clients (Section 7.1, 7.2).
NFR-05	Password Encryption	Passwords stored as salted bcrypt hashes (Section 7.1, 7.2).
NFR-06	Role-Based Restriction	RBAC middleware enforced on every API endpoint, scoped by role and branch (Section 7.1).
NFR-07	Audit Logs	Audit log table records key user actions with user ID, timestamp, and affected entity (Section 7.1).
(iii) Usability		
NFR ID	**Category**	**Design Decision Addressing the NFR**
NFR-08	Simple UI	Large tappable tiles, colour-coded statuses, and minimal-input forms across POS, KDS, and Inventory screens (Section 5.3).
NFR-09	Clear Navigation	Consistent, role-scoped navigation structure across all web screens, defined in the navigation flow diagram (Section 5.2).
NFR-10	Rider App Usability	Step-by-step guided delivery flow with integrated map navigation and single-action screens in the Rider App (Section 5.1.10, 5.1.11, 5.3).
(iv) Reliability		

NFR ID	Category	Design Decision Addressing the NFR
NFR-11	Offline Sync	POS and Rider App clients can queue actions locally (e.g., order creation, status updates) and synchronise with the API once connectivity is restored, with conflict handling defined at the API layer.
NFR-12	Data Consistency	All order, inventory, and delivery updates are performed as database transactions (e.g., deducting inventory and creating an order atomically) to maintain consistent records across branches.
NFR-13	Backup & Recovery	Scheduled automated database backups with a documented restore procedure (Section 7.2).
(v) Scalability		
NFR ID	**Category**	**Design Decision Addressing the NFR**
NFR-14	Branch Expansion	New branches are added as new Branch records with associated users, menu items, and inventory; no architectural change is required (Section 2.2, 3.1).
NFR-15	Load Scalability	Stateless API design allows horizontal scaling of API instances; Redis caching reduces database load as the number of users, orders, and inventory records grows (Section 3.1, 3.5).
(vi) Maintainability		
NFR ID	**Category**	**Design Decision Addressing the NFR**
NFR-16	Modular Architecture	Backend organised into independent feature modules (Authentication, Orders, Inventory, Delivery, Employees, Reporting), each with its own routes, controllers, and services (Section 3.1).
NFR-17	Coding Standards	Consistent project structure, linting, and code review practices are applied across modules; dependency versions are tracked and updated (Section 7.4, A06).
NFR-18	Non-Disruptive Updates	Stateless API instances allow rolling deployments (deploying new instances and retiring old ones) without taking the whole system offline.
(vii) Compatibility		
NFR ID	**Category**	**Design Decision Addressing the NFR**
NFR-19	Browser Support	React/Tailwind frontend built using standard, cross-browser-compatible web technologies, tested on Chrome, Edge, and Firefox (Section 6.1).
NFR-20	Android Support	Rider App built with Flutter, targeting Android smartphones with GPS capability (Section 3.5, 6.1).
(viii) Availability		

NFR ID	Category	Design Decision Addressing the NFR
NFR-21	Operating Hours	Cloud hosting on AWS/Railway.app with managed uptime, combined with offline-tolerant clients (NFR-11), minimises downtime during operating hours.
NFR-22	Continuous Operation	A single shared backend and database serve all branches continuously; horizontal scaling and rolling deployments (NFR-15, NFR-18) avoid system-wide outages during updates.